tapat

USER MANUAL

An exam monitor for Google Forms quizzes that blocks a listed set of sites and logs when students leave the exam.

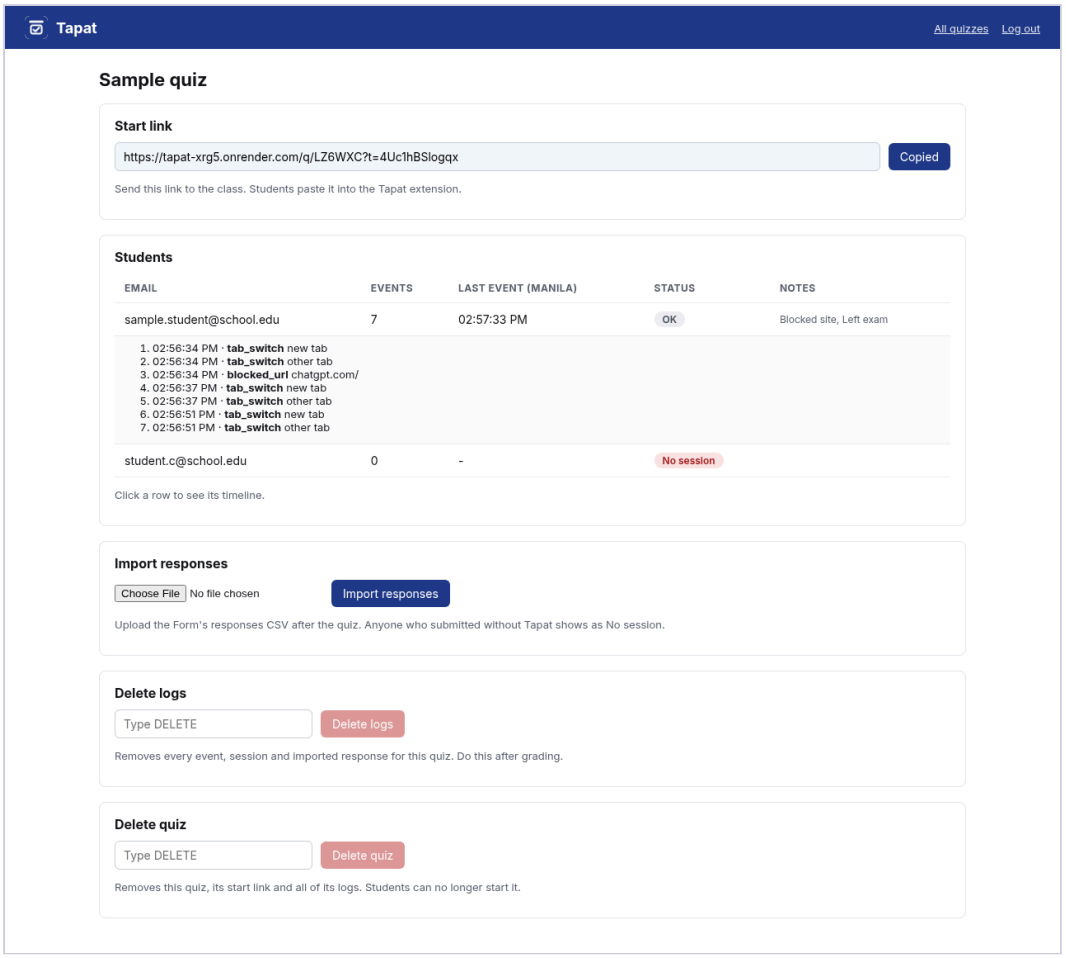
INTE301 Systems Integration & Architecture · Project 1

Sir Rodel Villareaz

Troy Lazaro

October 12, 2026

https://tapat-xrg5.onrender.com



Contents

1. What Tapat is and what it does
2. What it can do
3. How it works
4. Professor guide
5. Student guide
6. Status meanings and troubleshooting
7. Limitations and privacy

Appendix A. Deployment

Appendix B. Source code

- B1. Tapat server
- B2. Chrome extension
- B3. Desktop watcher
- B4. Documentation build

1. What Tapat is and what it does

Tapat is an exam monitor for quizzes made in Google Forms. *Tapat* is Filipino for "honest."

A student takes the quiz inside the Tapat Chrome extension instead of a normal browser tab. While the quiz is open, Tapat blocks a listed set of sites, notes when the student leaves the quiz, and sends those notes to the professor. The professor sees one row per student on a password-protected admin page and can look at each student's timeline after the quiz.

Tapat does not grade anything and does not decide who cheated. It gives the professor a record to look at, and the professor decides.

Live server: **https://tapat-xrg5.onrender.com**. Admin page: **https://tapat-xrg5.onrender.com/admin**.

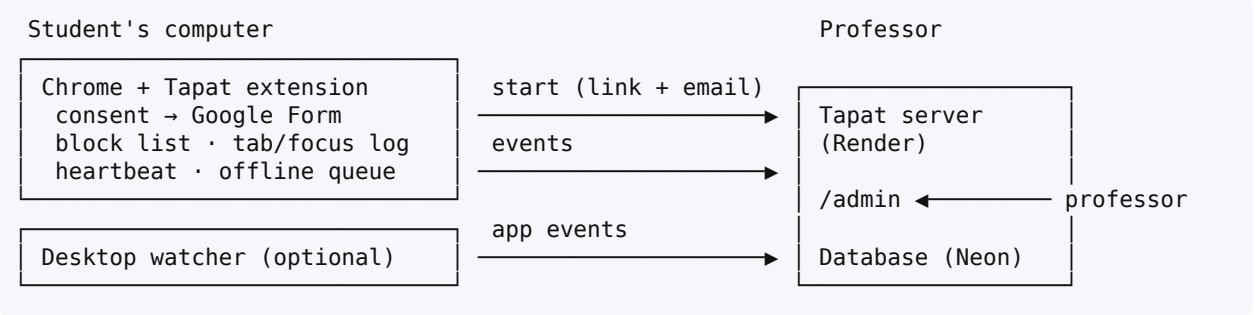
2. What it can do

Feature	What it does
Block list	Blocks a listed set of sites during the quiz: ChatGPT, Gemini, Claude, Word Online / office.com, Google Docs documents and any `.pdf` link. Google Forms itself is never blocked. Every blocked attempt is logged.
Focus tracking	Logs when the student switches tabs, opens a new tab, or Chrome loses focus because another app came to the front.
Submit detection	Notices when the Form shows "Your response has been recorded" and closes the session.
Heartbeat	The extension checks in every 10 seconds, so a monitor that stops is visible.
Offline queue	If the wifi drops, every event is kept in the browser with its own time and number and sent in order later. Nothing is lost or counted twice.
Waking state	The free server sleeps when idle. The popup shows "Server waking up..." and waits instead of failing.
No session (CSV)	After the quiz, the professor uploads the Form's responses CSV. Anyone who submitted the Form without Tapat running is marked No session.
Delete logs	One button removes every log for a quiz after grading.
Delete quiz	Removes a quiz, its start link and all of its logs.
Desktop watcher	An optional small program that logs when Word, Notes, Preview or Acrobat comes to the front. It records app names only, never window titles.

3. How it works

Tapat has three parts.

1. **The Chrome extension** runs on the student's computer. It shows the consent screen, opens the Google Form inside its own exam page, blocks the listed sites and records events.
2. **The Tapat server** runs on Render, with its data in a Postgres database on Neon. It keeps the quizzes and events and serves the admin page.
3. **The desktop watcher** (optional) runs on the student's computer next to Chrome and reports which watched app is in front.



Start links and session tokens. Each quiz has a start link with a secret part only the class receives. When a student starts, the server gives that student a session token tied to their email and that quiz. Every event must carry the token, so one student can't send events in a classmate's name.

Offline queue. Every event is saved in the browser first, then sent. If sending fails because the wifi is down or the server is still waking, the extension tries again after 2, 4, 8 and up to 60 seconds, and again as soon as the computer is back online. Events go out in their original order with the time they actually happened. The server ignores any event it already has.

Statuses. Each student row shows one status chip. They are explained in the Status meanings section.

4. Professor guide

Before the quiz

1. In the Google Form, open **Settings → Responses** and turn on **Collect email addresses**. Tapat matches students by email.
2. About a minute before the quiz, open https://tapat-xrg5.onrender.com/admin. The first load can take about a minute because the free server was asleep. That is normal.
3. Enter the admin password and click **Log in**.

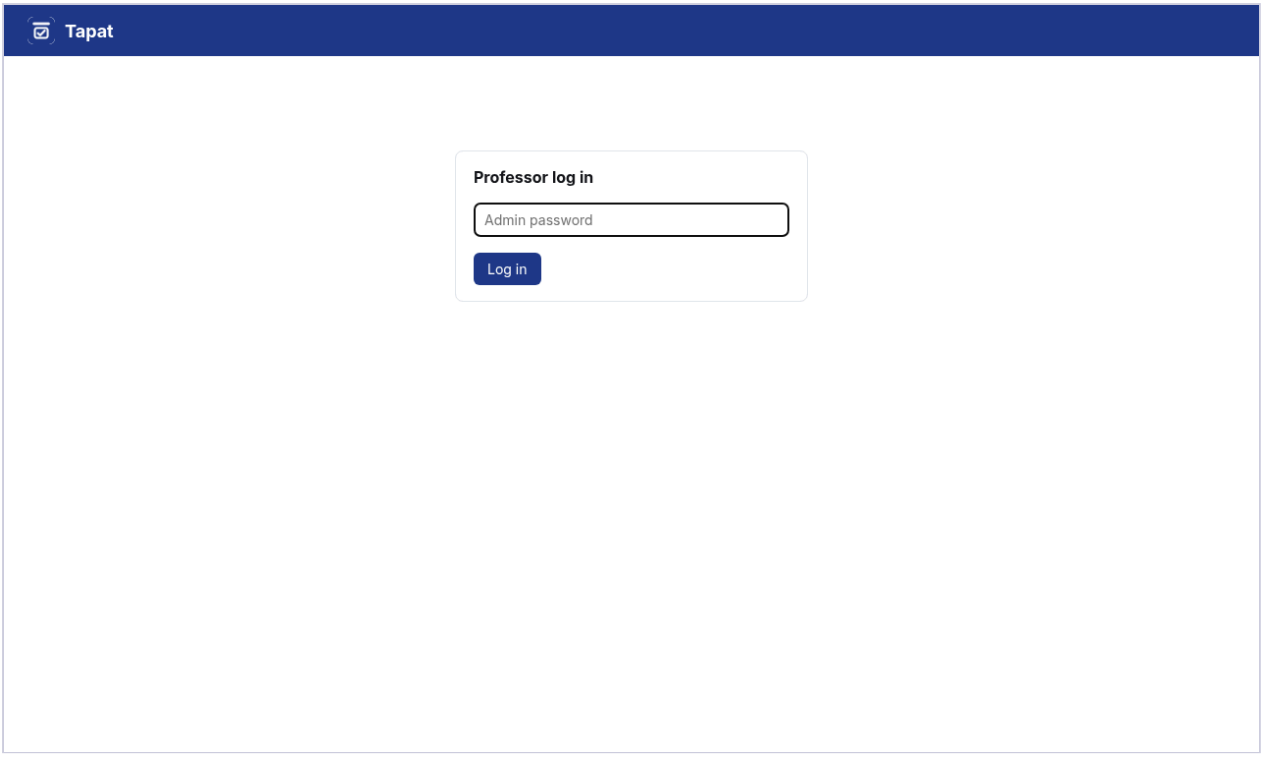


Figure 1. Professor log in. The password is set only in the server's environment.

1. Under New quiz, type a title, paste the Google Form link, and click **Create quiz**.

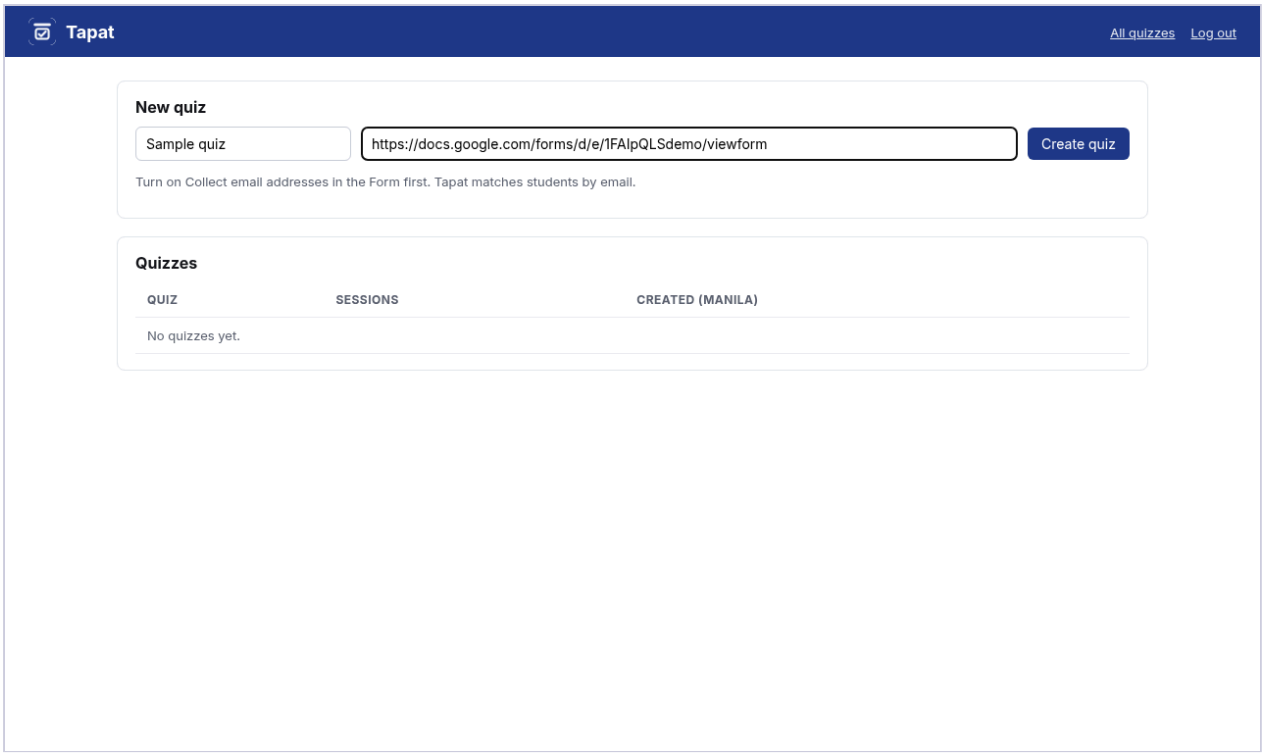


Figure 2. Admin home: create a quiz from a Google Form link, and the list of quizzes.

1. On the quiz page, click **Copy start link**. The button changes to **Copied**. Send the link to the class, for example in the class group chat or Google Classroom.

During the quiz

1. Watch the Students table. Each row is one student. Click a row to open that student's timeline. Click it again to close it.

After the quiz

1. In Google Forms, open **Responses**, click the three dots, and download the responses as CSV.
2. Under Import responses, choose the file and click **Import responses**. Anyone who submitted without Tapat appears with a red **No session** chip.
3. After grading, type `DELETE` in the box under Delete logs and click **Delete logs**. This removes every event, session and imported response for that quiz.
4. To remove the quiz itself, type `DELETE` in the box under Delete quiz and click **Delete quiz**. This also removes its start link and all of its logs, and students can no longer start it.

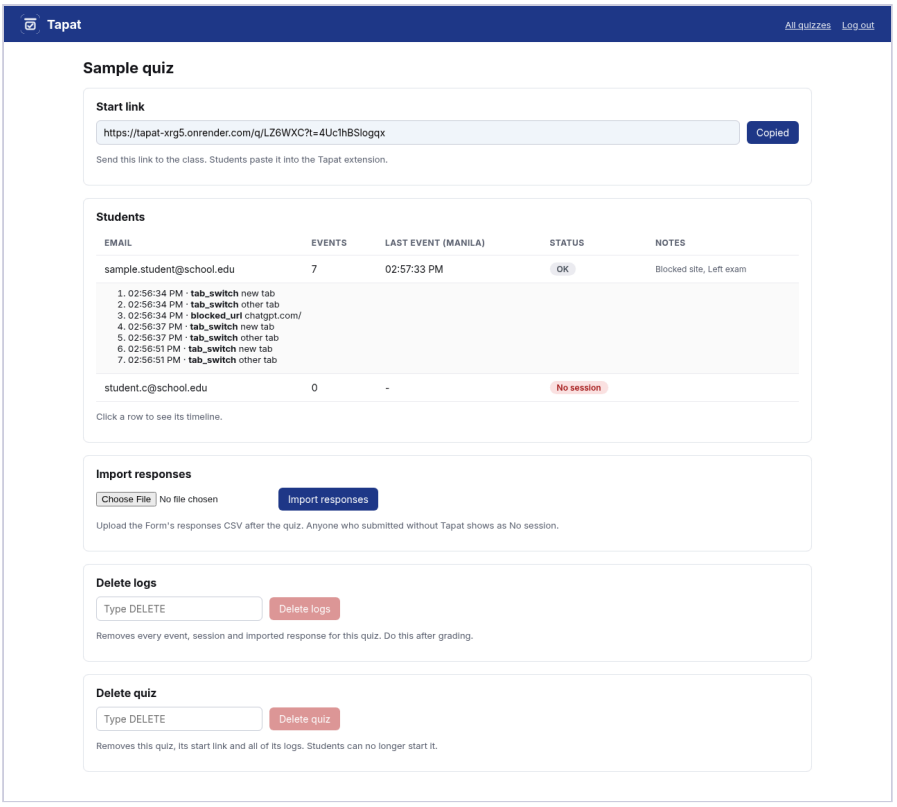


Figure 3. Admin quiz page on the live server: start link after Copy start link, a student with a blocked site and their timeline open, and a No session row after Import responses.

5. Student guide

Install the extension (once)

1. Download the Tapat extension folder your professor shared, and unzip it if needed.
2. In Chrome, open `chrome://extensions`.
3. Turn on **Developer mode** (top right).
4. Click **Load unpacked** and choose the `extension` folder.
5. Pin Tapat: click the puzzle piece icon in the toolbar, then the pin next to Tapat.

Take the quiz

1. Click the Tapat icon. Paste the start link from your professor, enter your school email (the same one you will use in the Form), and click **Open exam**.

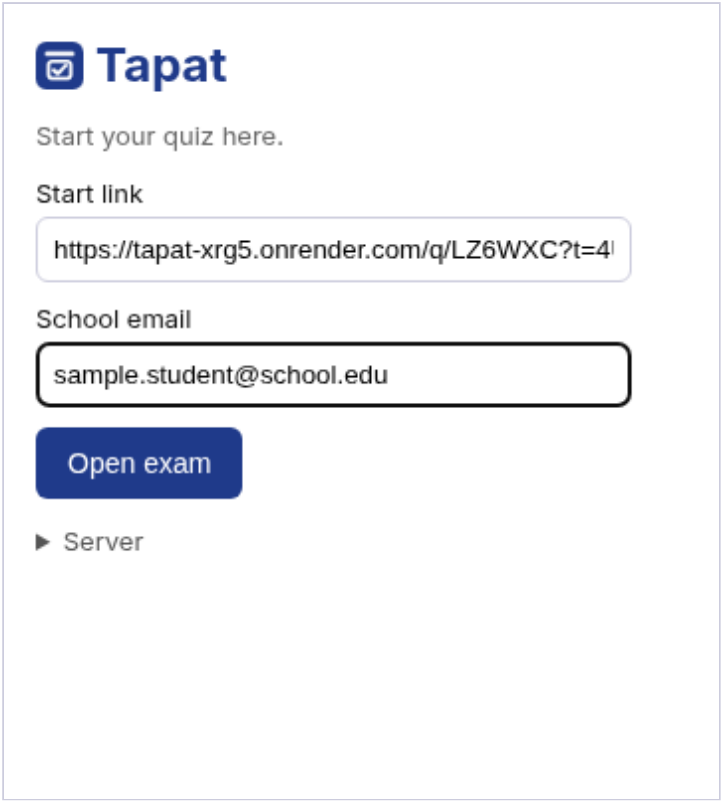


Figure 4. Extension popup: the student pastes the start link and enters their school email.

1. If the popup says "Server waking up...", wait. It can take a minute, and the exam opens by itself when the server answers.

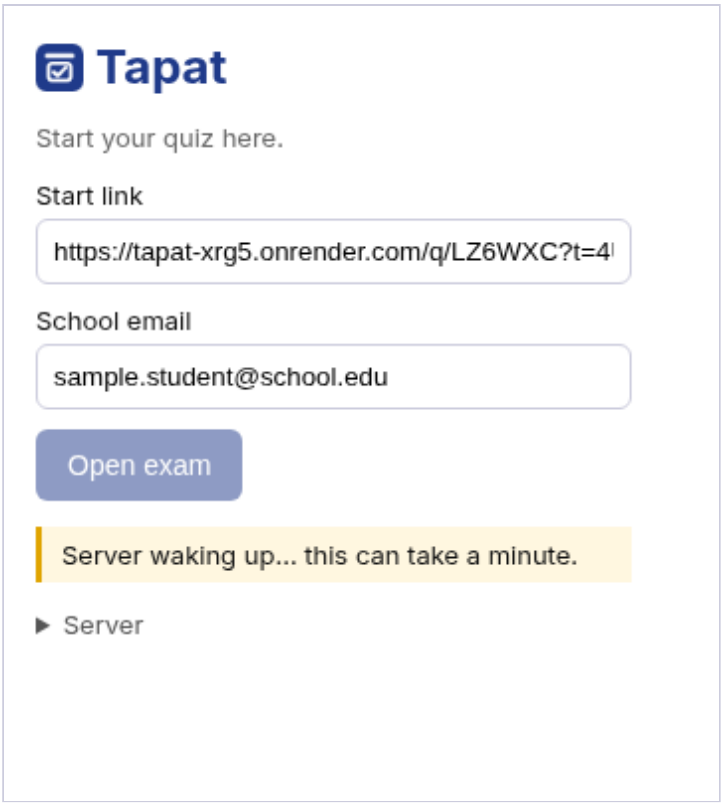


Figure 5. Popup while the free server wakes up. It waits and opens the exam once the server answers.

1. Read the consent screen and click **I understand, start the quiz**.

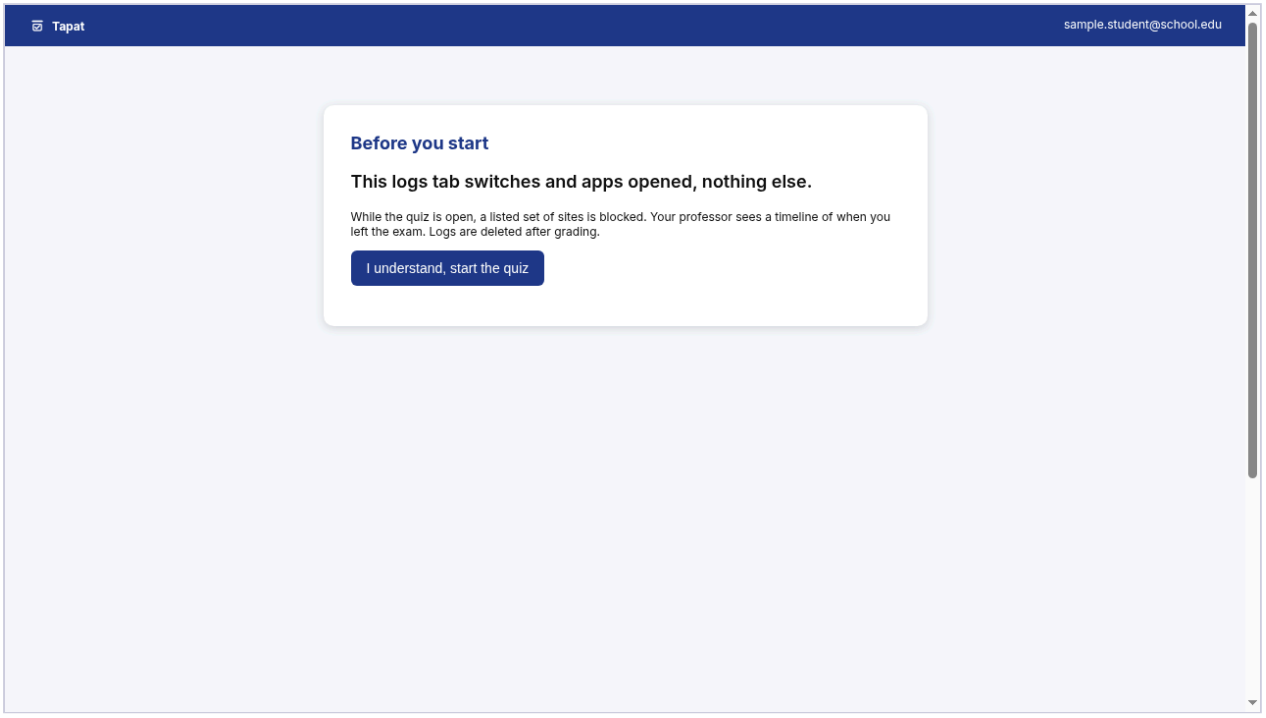


Figure 6. Consent screen shown before the quiz starts.

1. Answer the Form inside the exam page and submit it as usual.

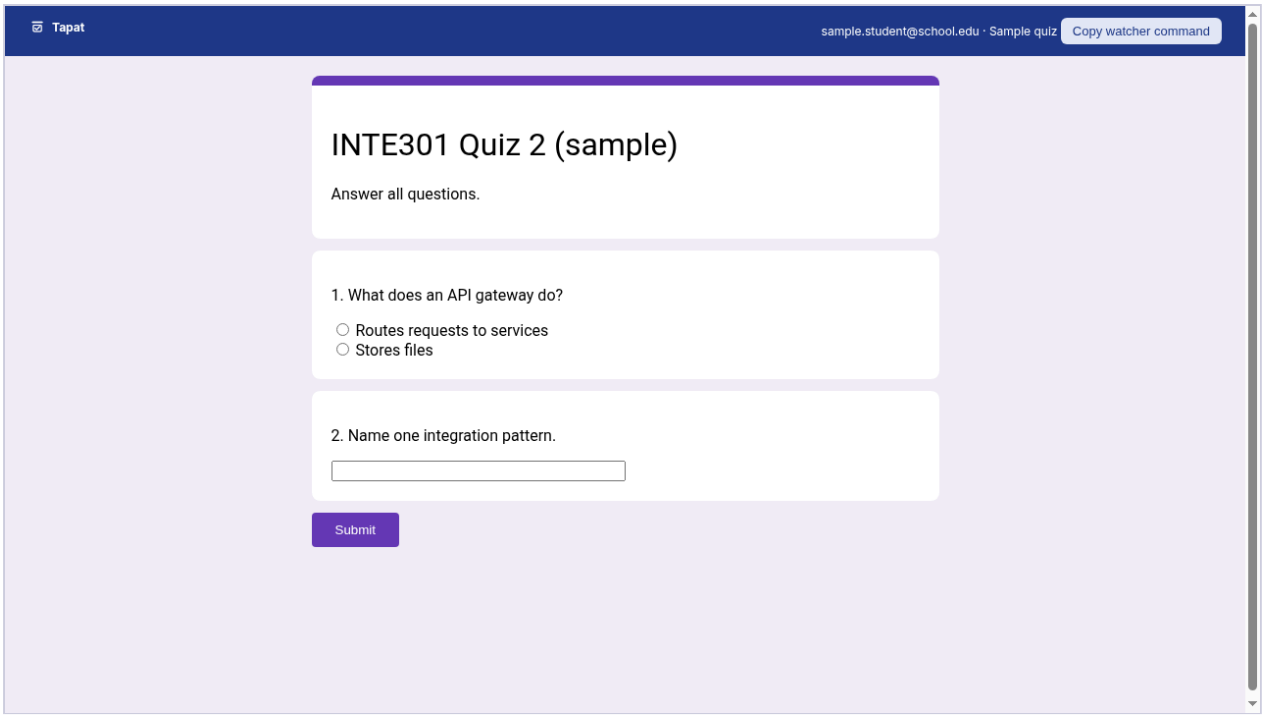


Figure 7. Exam page with a sample quiz embedded in place of a real Google Form.

1. If you open a site on the block list, you see the block page instead. Click **Back to the exam**. The attempt is logged.

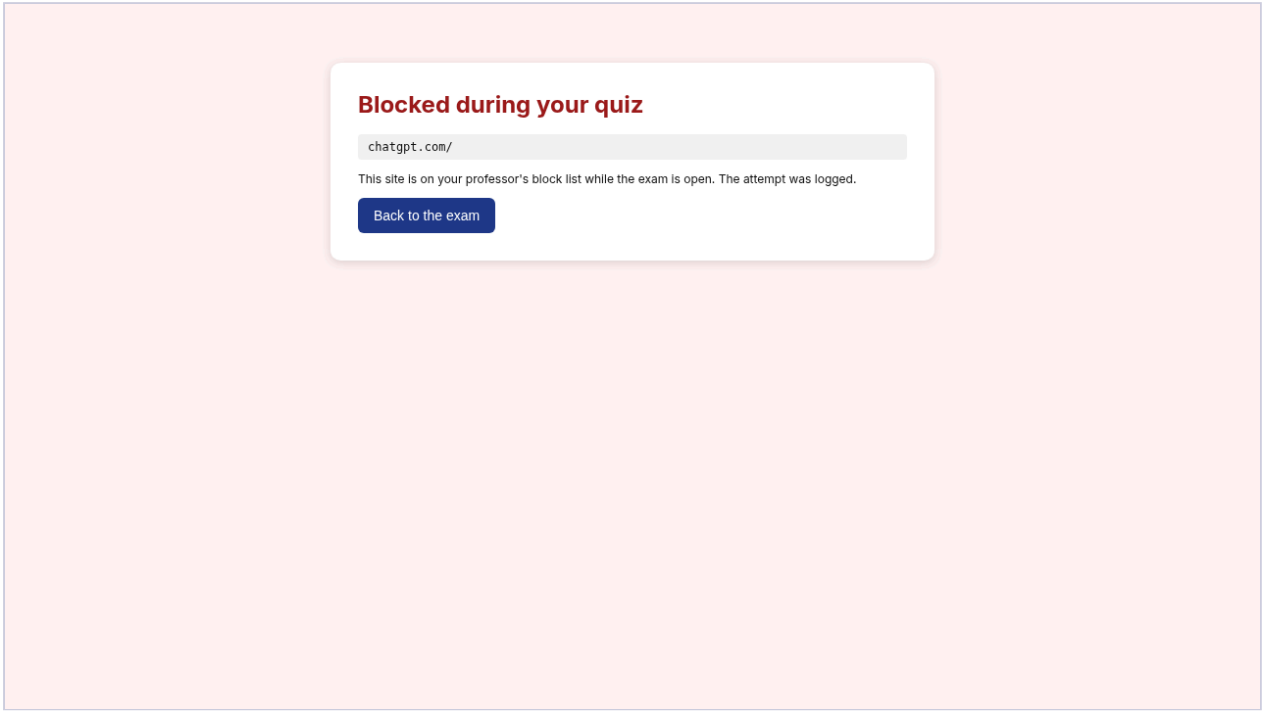


Figure 8. Block page shown when chatgpt.com is opened during the quiz; the attempt is logged.

1. If your wifi drops, keep answering. The popup shows "Offline, N events queued", and the events are sent once you are back online.

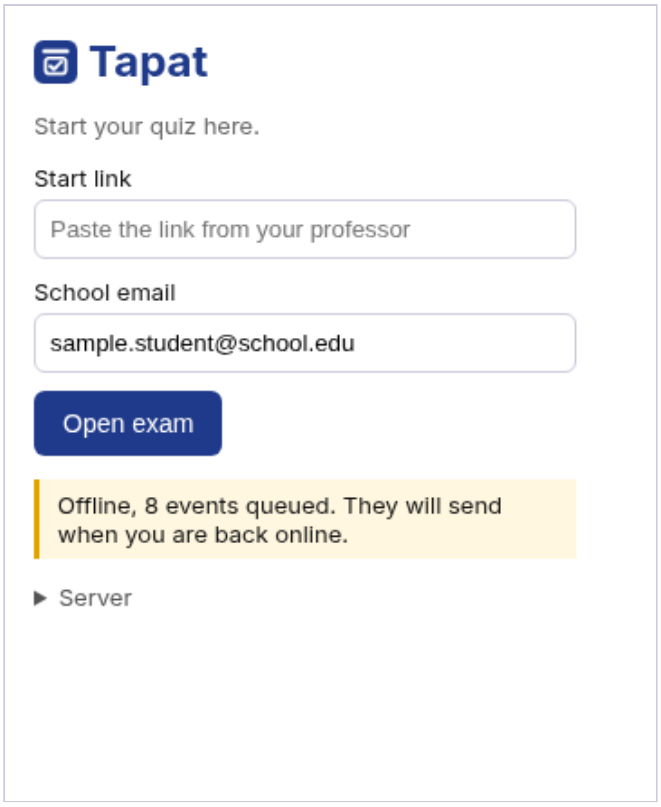


Figure 9. Popup while the server can't be reached: events are kept in the browser and sent in order later.

1. When the Form says your response has been recorded, Tapat stops. You can close the tab.

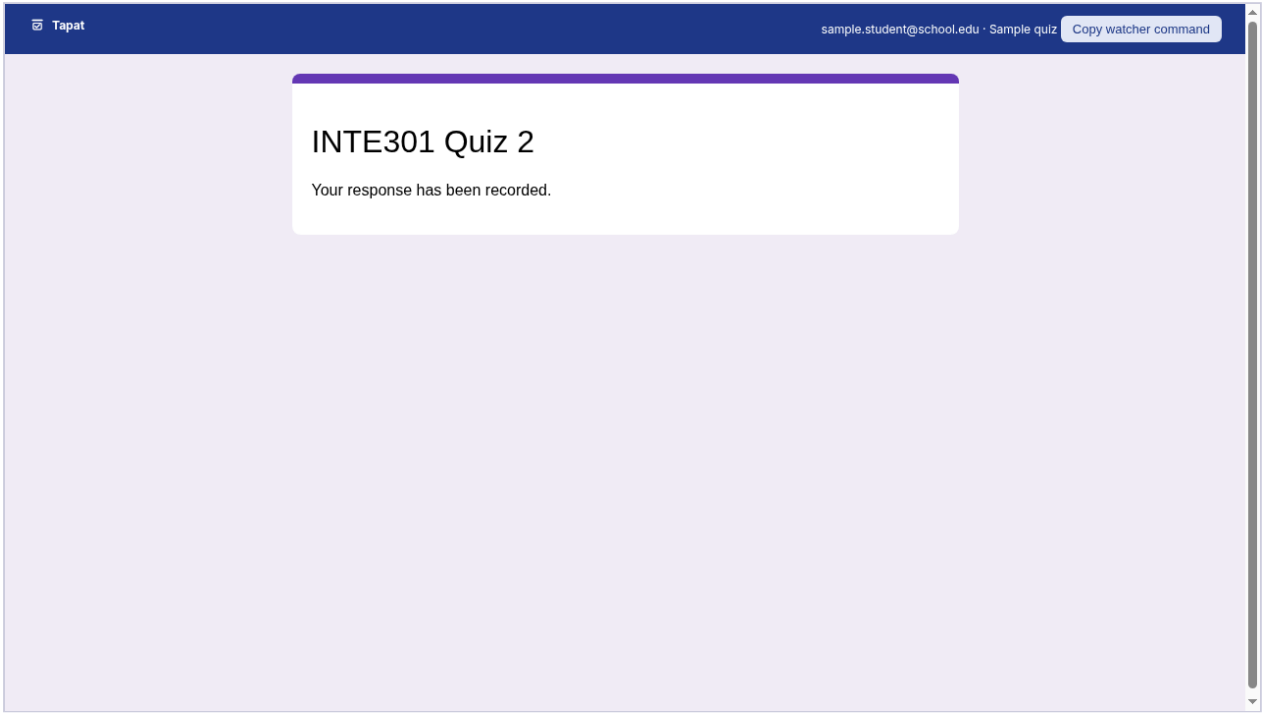


Figure 10. Sample quiz submitted: Tapat detects the formResponse page and closes the session.

Optional, the desktop watcher. If your professor asks for it, click **Copy watcher command** at the top of the exam page and run it in a terminal from the `watcher` folder. It stops by itself when you submit.

6. Status meanings and troubleshooting

Status	Color	Meaning
OK	Gray	Tapat ran the whole time and the heartbeats had no long gaps.
Offline gap, synced later	Gray-amber	The student's connection dropped for a while. The events from that time arrived later, in order, so nothing is missing.
Monitor silent	Amber	No heartbeat for more than 25 seconds and nothing filled the gap later. The extension may have been turned off, Chrome closed, or the browser data cleared.
No session	Red	The student is in the Form responses CSV but never ran Tapat.

The Notes column lists what happened during the session: **Blocked site**, **App opened**, and **Left exam** (three or more tab switches or focus losses).

The admin page takes a minute to load. The free server sleeps after about 15 minutes without use. Open the admin page about a minute before the quiz. Once students start, their heartbeats keep it awake.

A student says the popup is stuck on "Server waking up...". Wait up to a minute. If it gives up, check the internet connection and click **Open exam** again.

A student lost wifi. Nothing needs to be done. Their events are kept and sent later, and the row shows Offline gap, synced later.

A student shows No session. They submitted the Form without starting Tapat, for example in another browser or an incognito window, or they typed a different email in Tapat than in the Form. Ask them about it.

A student shows Monitor silent. Open their timeline to see when the heartbeats stopped, and ask them what happened.

The CSV import found 0 responses. Collect email addresses was off in the Form, so the CSV has no email column. Turn it on before the next quiz.

7. Limitations and privacy

Limitations

- It blocks a list, not every site. A site that isn't on the list stays open, and a second device can't be seen at all.
- Each session has its own token, so events can't be faked for a classmate. It doesn't prove the student typed their own email. The responses CSV catches that mismatch afterward.
- Events queued offline are kept in the browser. If the student clears browser data or removes the extension before reconnecting, those events are lost and the gap shows as Monitor silent.
- The watcher only runs if the student starts it, and it stops on submit.
- Students who decline take the quiz on paper or in a proctored room.
- A flag is a reason to ask, not proof of cheating. The professor decides.

Privacy

Events are stored on a hosted server (Render) with a database on Neon, and only the teacher can see them through a password-protected admin page. Tapat stores emails and event types only, never page contents, keystrokes or window titles. Logs are deleted after grading with **Delete logs**.

Appendix A. Deployment

Tapat runs on a free Render web service with a free Postgres database on Neon. No card is needed for either. Render's free disk is wiped on every restart, which is why the data goes to Neon instead of a local file.

Time needed: about 15 minutes.

1. Create the database on Neon

1. Go to https://neon.tech and click **Sign up**. Sign in with GitHub or Google.
2. Create a project:
3. Project name: `tapat`
4. Postgres version: leave the default
5. Region: **AWS Asia Pacific (Singapore)**, the closest to Manila
6. Click **Create project**.
7. On the project dashboard, click **Connect**.
8. Leave Branch `main`, Database `neondb` and the default role as they are. Turn **Connection pooling** off, so the string has no `-pooler` in the host.
9. Copy the connection string. It looks like: `postgresql://neondb_owner:XXXX@ep-something-123456.ap-southeast-1.aws.neon.tech/neondb?sslmode=require` Keep it somewhere private for step 2. Do not commit it.

Tapat creates its tables on first start, so there is nothing else to do in Neon.

2. Create the service on Render with the Blueprint

1. Sign in at https://dashboard.render.com with the existing account.
2. Click **New** (top right), then **Blueprint**.
3. If GitHub isn't connected for this repo yet, click **Configure account** and give Render access to `Troy-LL/tapat`.
4. Next to `Troy-LL/tapat`, click **Connect**.
5. Blueprint name: `tapat`. Branch: `main`. Leave Blueprint Path empty (it uses `render.yaml`).
6. Render lists one new resource, the web service `tapat` on the **Free** plan, and asks for two values:
7. `DATABASE_URL`: paste the Neon connection string from step 1.
8. `ADMIN_PASSWORD`: choose the professor's password. It is stored only in Render, never in the repo. Share it with the professor privately. `SECRET_KEY` is generated by Render automatically.
9. Click **Deploy Blueprint**. The first build takes a few minutes.
10. When the service shows **Live**, open it. The address is shown under the service name, for example `https://tapat-xrg5.onrender.com`. Check https://tapat-xrg5.onrender.com/health shows `{"ok":true}`.

Render only asks for `DATABASE_URL` and `ADMIN_PASSWORD` the first time. To change them later, open the service, click **Environment**, edit the value, and click **Save, rebuild, and deploy**.

3. Point the extension at the server

The live server is `https://tapat-xrg5.onrender.com`, and the extension already points there (`DEFAULT_SERVER` in `extension/config.js` and `host_permissions` in `extension/manifest.json`).

If the service is ever recreated with a different address:

1. In `extension/config.js`, set `DEFAULT_SERVER` to the new address.
2. In `extension/manifest.json`, replace `https://tapat-xrg5.onrender.com/*` under `host_permissions` with the new address followed by `/*`.
3. Commit, then reload the extension in `chrome://extensions`.

A start link always carries its own server address, so students who paste the full start link reach the right server either way.

4. Share the extension with the class

1. Zip the `extension/` folder, or share the repo link.
2. Students open `chrome://extensions`, turn on **Developer mode**, click **Load unpacked**, and pick the `extension/` folder.

5. Quiz day

- About a minute before the quiz, open https://tapat-xrg5.onrender.com/admin and log in. The free service sleeps after 15 minutes without traffic, and the first load can take about a minute. That is normal.
- Once students start, the 10-second heartbeats keep the server awake.
- If a student starts while it is still waking, the popup shows "Server waking up..." and waits. Events are queued in the browser and sent once the server answers, so nothing is lost.

Free tier notes

- Render free web services sleep when idle and have a monthly limit of free instance hours. One class's quizzes fit well inside it.
- Neon's free plan includes a small amount of storage, far more than Tapat's event logs need. Use **Delete logs** after grading.
- No paid add-ons are needed.

Appendix B1. Source code: Tapat server

9 files.

server/app.py · Python · 433 lines

```python
"""Tapat server: receives monitor events and serves the professor's admin pages."""
import csv
import os
import io
import hmac
import secrets
import time
from datetime import datetime, timedelta, timezone
from html import escape

from fastapi import Cookie, FastAPI, File, Form, HTTPException, Query, Request, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse, JSONResponse, RedirectResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

import auth
import db

SILENT_GAP = 25  # seconds without a heartbeat before "Monitor silent"
EVENT_TYPES = {"tab_switch", "focus_lost", "blocked_url", "app_opened", "heartbeat", "submitted"}
MANILA = timezone(timedelta(hours=8))
CODE_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"

app = FastAPI(title="Tapat")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["GET", "POST"], allow_headers=["*"])
db.init()
app.mount("/static", StaticFiles(directory=os.path.join(os.path.dirname(__file__), "static")), name="static")


# ---------- student and extension API ----------

class StartRequest(BaseModel):
    code: str
    token: str
    email: str


class Event(BaseModel):
    session_id: str
    session_token: str
    type: str
    seq: int  # per-session counter from the extension; resends are ignored
    student: str = ""  # informational; the server uses the email bound to the session
    detail: str = ""
    ts: float | None = None  # client time when it happened, kept even if sent much later


@app.get("/health")
def health():
    return {"ok": True}


@app.post("/api/start")
```

```python
def start_session(req: StartRequest):
    """Start link (quiz code + quiz token) plus email -> a session token for this student only."""
    q = get_quiz(req.code)
    if not q or not hmac.compare_digest(req.token.encode(), q["token"].encode()):
        return JSONResponse({"ok": False, "error": "Start link is not valid"}, status_code=403)
    email = req.email.strip().lower()
    if "@" not in email:
        return JSONResponse({"ok": False, "error": "Enter your school email"}, status_code=400)
    sid, tok = secrets.token_hex(12), secrets.token_urlsafe(24)
    with db.connect() as c:
        c.execute("INSERT INTO sessions(id, token, quiz, email, created) VALUES (?,?,?,?,?)",
                  (sid, tok, q["code"], email, time.time()))
    return {"ok": True, "session_id": sid, "session_token": tok, "form_url": q["form_url"],
            "title": q["title"], "quiz": q["code"]}


def get_session(sid: str, token: str):
    with db.connect() as c:
        s = c.execute("SELECT * FROM sessions WHERE id=?", (sid,)).fetchone()
    if s and hmac.compare_digest(token.encode(), s["token"].encode()):
        return s
    return None


@app.post("/events")
def post_event(ev: Event):
    if ev.type not in EVENT_TYPES:
        return JSONResponse({"ok": False, "error": f"unknown type {ev.type}"}, status_code=400)
    s = get_session(ev.session_id, ev.session_token)
    if not s:
        return JSONResponse({"ok": False, "error": "bad session token"}, status_code=403)
    now = time.time()
    with db.connect() as c:
        cur = c.execute(
            "INSERT INTO events(quiz, session_id, seq, student, type, detail, ts, received) VALUES (?,?,?,?,?,?,?,?) "
            "ON CONFLICT (session_id, seq) DO NOTHING",
            (s["quiz"], s["id"], ev.seq, s["email"], ev.type, ev.detail[:300], ev.ts or now, now),
        )
    return {"ok": True, "duplicate": cur.rowcount == 0}


@app.get("/sessions/{session_id}")
def session_state(session_id: str):
    """Polled by the desktop watcher; 'closed' once the Form is submitted."""
    with db.connect() as c:
        done = c.execute("SELECT 1 FROM events WHERE session_id=? AND type='submitted' LIMIT 1", (session_id,)).fetchone()
        known = c.execute("SELECT 1 FROM sessions WHERE id=?", (session_id,)).fetchone()
    return {"session_id": session_id, "state": "closed" if done else ("open" if known else "unknown")}


@app.get("/q/{code}", response_class=HTMLResponse)
```

```
106  def student_start(code: str, request: Request, t: str = ""):
107      q = get_quiz(code)
108      if not q or not hmac.compare_digest(t.encode(), q["token"].encode()):
109          return page("Quiz not found", "<div class=card><h2>This start link is not v
alid.</h2>"
110                      "<p>Ask your professor for the link again.</p></div>", 404)
111      body = f"""<div class=card style="max-width:620px"><h2>{escape(q['title'])}</h2
>
112  <p>This quiz is taken through the Tapat Chrome extension.</p>
113  <ol><li>Install the Tapat extension your professor shared.</li>
114  <li>Copy this page's address from the address bar.</li>
115  <li>Click the Tapat icon, paste the link under <b>Start link</b>, enter your school
email, and click <b>Open exam</b>.</li></ol>
116  <p class=muted>Use the same email you will use in the Google Form.</p></div>"""
117      return page(q["title"], body)
118
119
120  # ---------- data ----------
121
122  def get_quiz(code: str):
123      with db.connect() as c:
124          return c.execute("SELECT * FROM quizzes WHERE code=?", (code.upper(),)).fet
chone()
125
126
127  def new_code() -> str:
128      while True:
129          code = "".join(secrets.choice(CODE_ALPHABET) for _ in range(6))
130          if not get_quiz(code):
131              return code
132
133
134  def create_quiz(title: str, form_url: str) -> str:
135      code = new_code()
136      with db.connect() as c:
137          c.execute("INSERT INTO quizzes(code, token, title, form_url, created) VALUE
S (?,?,?,?,?)",
138                    (code, secrets.token_urlsafe(9), title.strip() or "Quiz", form_ur
l.strip(), time.time()))
139      return code
140
141
142  def import_rows(quiz: str, text: str) -> int:
143      """Read the Google Form responses CSV. Needs 'Collect email addresses' on."""
144      reader = csv.DictReader(io.StringIO(text))
145      email_col = next((h for h in (reader.fieldnames or []) if "email" in h.lower
()), None)
146      if not email_col:
147          return 0
148      ts_col = next((h for h in reader.fieldnames if "timestamp" in h.lower()), None)
149      n = 0
150      with db.connect() as c:
151          for r in reader:
152              email = (r.get(email_col) or "").strip().lower()
153              if email:
154                  c.execute(
155                      "INSERT INTO responses(quiz, email, submitted_at) VALUES
(?,?,?) "
156                      "ON CONFLICT (quiz, email) DO UPDATE SET submitted_at = exclude
d.submitted_at",
157                      (quiz, email, r.get(ts_col, "") if ts_col else ""),
158                  )
159                  n += 1
```

```
160      return n
161
162
163  def delete_logs(quiz: str):
164      with db.connect() as c:
165          c.execute("DELETE FROM events WHERE quiz=?", (quiz,))
166          c.execute("DELETE FROM responses WHERE quiz=?", (quiz,))
167          c.execute("DELETE FROM sessions WHERE quiz=?", (quiz,))
168
169
170  def summarize(quiz: str, now: float | None = None):
171      now = now or time.time()
172      with db.connect() as c:
173          events = c.execute("SELECT * FROM events WHERE quiz=? ORDER BY ts, seq", (q
uiz,)).fetchall()
174          responses = {r["email"]: r["submitted_at"] for r in c.execute("SELECT * FRO
M responses WHERE quiz=?", (quiz,))}
175      students: dict[str, dict] = {}
176      for e in events:
177          s = students.setdefault(e["student"], {
178              "email": e["student"], "counts": {t: 0 for t in EVENT_TYPES}, "timelin
e": [],
179              "last_beat": None, "first": e["ts"], "submitted": False, "silent_gaps":
0, "offline_gaps": 0, "late_since_beat": False, "n_events": 0,
180          })
181          s["last"] = e["ts"]
182          late = (e["received"] or e["ts"]) - e["ts"] > SILENT_GAP
183          if e["type"] == "heartbeat":
184              if s["last_beat"] and e["ts"] - s["last_beat"] > SILENT_GAP:
185                  gap = int(e["ts"] - s["last_beat"])
186                  # A gap closed by events that arrived late was the student being
187                  # offline: the extension queued everything and sent it afterward.
188                  if late or s["late_since_beat"]:
189                      s["offline_gaps"] += 1
190                      s["timeline"].append((s["last_beat"], "offline_gap", f"offline
for {gap}s, synced later"))
191                  else:
192                      s["silent_gaps"] += 1
193                      s["timeline"].append((s["last_beat"], "monitor_silent", f"no he
artbeat for {gap}s"))
194              s["last_beat"] = e["ts"]
195              s["late_since_beat"] = False
196          else:
197              s["timeline"].append((e["ts"], e["type"], e["detail"]))
198              s["n_events"] += 1
199              s["late_since_beat"] = s.get("late_since_beat") or late
200          s["counts"][e["type"]] += 1
201          if e["type"] == "submitted":
202              s["submitted"] = True
203      rows = []
204      for s in students.values():
205          s["timeline"].sort(key=lambda item: item[0])
206          flags = []
207          if s["counts"]["blocked_url"]:
208              flags.append("Blocked site")
209          if s["counts"]["app_opened"]:
210              flags.append("App opened")
211          if s["counts"]["tab_switch"] + s["counts"]["focus_lost"] >= 3:
212              flags.append("Left exam")
213          live_gap = (not s["submitted"]) and s["last_beat"] and now - s["last_beat"]
> SILENT_GAP
214          if s["silent_gaps"] or live_gap:
215              flags.append("Monitor silent")
```

```
216          elif s["offline_gaps"]:
217              flags.append("Offline gap, synced later")
218          s["flags"] = flags
219          rows.append(s)
220      for email in responses:
221          if email not in students:
222              rows.append({"email": email, "counts": {t: 0 for t in EVENT_TYPES}, "ti
meline": [],
223                           "flags": ["No session"], "submitted": True, "first": None,
"last": None, "n_events": 0})
224      return sorted(rows, key=lambda r: r["email"])
225
226
227  # ---------- HTML ----------
228
229  def fmt(ts, full=False):
230      if not ts:
231          return "-"
232      return datetime.fromtimestamp(ts, MANILA).strftime("%b %d, %I:%M:%S %p" if full
else "%I:%M:%S %p")
233
234
235  ACCENT = "#1F3A8A"
236  CSS = """
237  *{box-sizing:border-box}
238  body{font-family:Inter,Segoe UI,Helvetica,Arial,sans-serif;margin:0;background:#ff
f;color:#14161a}
239  header{background:#1F3A8A;color:#fff;padding:12px 24px;display:flex;justify-conten
t:space-between;align-items:center}
240  header .brand{display:flex;align-items:center;gap:10px;font-weight:700;font-size:18
px;color:#fff;text-decoration:none}
241  header .brand img{height:28px;width:28px;border-radius:6px;background:#fff}
242  header nav a{color:#fff;font-size:13px;margin-left:16px;opacity:.9}
243  main{padding:24px;max-width:1100px;margin:0 auto}
244  h2{font-size:16px;margin:0 0 10px;color:#14161a}
245  section{border:1px solid #e3e6ee;border-radius:8px;padding:16px 18px;margin-bottom:
18px}
246  button,.btn{background:#1F3A8A;color:#fff;border:0;border-radius:6px;padding:8px 14
px;font-size:14px;cursor:pointer;font-family:inherit}
247  button.danger{background:#b91c1c}button:disabled{opacity:.45;cursor:not-allowed}
248  input[type=text],input[type=url],input[type=password]{padding:8px 10px;border:1px s
olid #c9cedb;border-radius:6px;font-size:14px;font-family:inherit}
249  .row{display:flex;gap:10px;align-items:center;flex-wrap:wrap}
250  .link{flex:1;min-width:280px;font-family:Consolas,monospace;font-size:13px;backgrou
nd:#f4f6fa}
251  .muted{color:#5b6170;font-size:13px}.err{color:#b91c1c}
252  table{width:100%;border-collapse:collapse}
253  th,td{padding:10px 12px;text-align:left;font-size:14px;border-bottom:1px solid #ece
ff4}
254  th{font-size:12px;text-transform:uppercase;letter-spacing:.04em;color:#5b6170}
255  tr.student{cursor:pointer}tr.student:hover{background:#f6f8fc}
256  .chip{display:inline-block;padding:2px 10px;border-radius:999px;font-size:12px;font
-weight:600}
257  .chip.ok{background:#eceef2;color:#4a505c}.chip.silent{background:#fdf0d5;color:#8a
5a00}.chip.offline{background:#f3ecdc;color:#6e5a2e}.chip.none{background:#fde2e2;color:#
a61b1b}
258  .notes{color:#5b6170;font-size:12px}
259  tr.tl td{background:#fafbfd;padding:6px 12px 12px 28px}tr.tl ol{margin:0;padding-le
ft:18px;font-size:13px}
260  .login{max-width:360px;margin:72px auto}
261  .list a{color:#1F3A8A}
262  """
263
```

```
 264
 265  def page(title, body, status=200, nav=""):
 266      html = f"""<!doctype html><html lang=en><head><meta charset=utf-8><meta name=vi
ewport content="width=device-width">
 267  <title>{escape(title)} · Tapat</title><style>{CSS}</style></head>
 268  <body><header><a class=brand href="/admin"><img src="/static/mark.png" alt="">Tapat
</a><nav>{nav}</nav></header>
 269  <main>{body}</main></body></html>"""
 270      return HTMLResponse(html, status_code=status)
 271
 272
 273  ADMIN_NAV = '<a href="/admin">All quizzes</a><a href="/admin/logout">Log out</a>'
 274
 275
 276  def require_admin(cookie):
 277      if not auth.valid_cookie(cookie):
 278          raise HTTPException(303, headers={"Location": "/admin/login"})
 279
 280
 281  def start_link(request: Request, quiz) -> str:
 282      return str(request.base_url).rstrip("/") + f"/q/{quiz['code']}?t={quiz['toke
n']}"
 283
 284
 285  @app.get("/", include_in_schema=False)
 286  def root():
 287      return RedirectResponse("/admin")
 288
 289
 290  @app.get("/admin/login", response_class=HTMLResponse)
 291  def login_page(error: str = ""):
 292      msg = {"1": "Wrong password.", "rate": "Too many tries. Wait 10 minutes and try
again."}.get(error, "")
 293      if not auth.admin_password():
 294          msg = "ADMIN_PASSWORD is not set on the server."
 295      err = f"<p class=err>{msg}</p>" if msg else ""
 296      return page("Log in", f"""<section class=login><h2>Professor log in</h2>{err}
 297  <form method=post action=/admin/login><p><input type=password name=password placeho
lder="Admin password" autofocus style="width:100%"></p>
 298  <button>Log in</button></form></section>""")
 299
 300
 301  @app.post("/admin/login")
 302  def login(request: Request, password: str = Form("")):
 303      # Behind Render's proxy the real address is the last X-Forwarded-For entry.
 304      fwd = request.headers.get("x-forwarded-for", "")
 305      ip = fwd.split(",")[-1].strip() if fwd else (request.client.host if request.cli
ent else "?")
 306      if auth.too_many_tries(ip):
 307          return RedirectResponse("/admin/login?error=rate", status_code=303)
 308      if not auth.check_password(password):
 309          auth.record_fail(ip)
 310          return RedirectResponse("/admin/login?error=1", status_code=303)
 311      auth.clear_fails(ip)
 312      r = RedirectResponse("/admin", status_code=303)
 313      r.set_cookie(auth.COOKIE, auth.make_cookie(), max_age=auth.MAX_AGE, httponly=Tr
ue, samesite="lax",
 314                   secure=request.url.scheme == "https")
 315      return r
 316
 317
 318  @app.get("/admin/logout")
 319  def logout():
```

```
    r = RedirectResponse("/admin/login", status_code=303)
    r.delete_cookie(auth.COOKIE)
    return r


@app.get("/admin", response_class=HTMLResponse)
def admin_home(tapat_admin: str | None = Cookie(None)):
    require_admin(tapat_admin)
    with db.connect() as c:
        quizzes = c.execute("SELECT * FROM quizzes ORDER BY created DESC").fetchall()
        counts = {r["quiz"]: r["n"] for r in c.execute("SELECT quiz, COUNT(*) AS n FROM sessions GROUP BY quiz")}
    rows = "".join(
        f"<tr><td><a href='/admin/quiz/{q['code']}'>{escape(q['title'])}</a></td>"
        f"<td>{counts.get(q['code'], 0)}</td><td>{fmt(q['created'], True)}</td></tr>" for q in quizzes
    ) or "<tr><td colspan=3 class=muted>No quizzes yet.</td></tr>"
    body = f"""<section><h2>New quiz</h2>
 <form method=post action=/admin/quiz class=row><input type=text name=title placeholder="Quiz title" required style="width:220px">
 <input type=url name=form_url placeholder="Google Form link" required style="flex:1;min-width:280px"><button>Create quiz</button></form>
 <p class=muted>Turn on Collect email addresses in the Form first. Tapat matches students by email.</p></section>
 <section class=list><h2>Quizzes</h2><table><tr><th>Quiz</th><th>Sessions</th><th>Created (Manila)</th></tr>{rows}</table></section>"""
    return page("Admin", body, nav=ADMIN_NAV)


@app.post("/admin/quiz")
def admin_create(title: str = Form(...), form_url: str = Form(...), tapat_admin: str | None = Cookie(None)):
    require_admin(tapat_admin)
    if not form_url.strip().startswith("https://docs.google.com/forms/"):
        raise HTTPException(400, "Paste a Google Forms link (https://docs.google.com/forms/...)")
    code = create_quiz(title, form_url)
    return RedirectResponse(f"/admin/quiz/{code}", status_code=303)


def status_chip(flags):
    if "No session" in flags:
        return '<span class="chip none">No session</span>'
    if "Monitor silent" in flags:
        return '<span class="chip silent">Monitor silent</span>'
    if "Offline gap, synced later" in flags:
        return '<span class="chip offline">Offline gap, synced later</span>'
    return '<span class="chip ok">OK</span>'


@app.get("/admin/quiz/{code}", response_class=HTMLResponse)
def admin_quiz(code: str, request: Request, imported: int | None = None, deleted: int = 0,
               tapat_admin: str | None = Cookie(None)):
    require_admin(tapat_admin)
    quiz = get_quiz(code)
    if not quiz:
        raise HTTPException(404, "No quiz with that code")
    code = quiz["code"]
    trs = []
    for i, r in enumerate(summarize(code)):
        notes = ", ".join(f for f in r["flags"] if f not in ("No session", "Monitor
```

```
silent", "Offline gap, synced later"))
 373          tl = "".join(f"<li>{fmt(t)} · <b>{escape(k)}</b> {escape(d)}</li>" for t,
k, d in r["timeline"]) \
 374              or "<li class=muted>No events. This student submitted the Form without
Tapat.</li>"
 375          trs.append(
 376              f"<tr class=student onclick=\"document.getElementById('tl{i}').hidden^=
1\"><td>{escape(r['email'])}</td>"
 377              f"<td>{r['n_events']}</td><td>{fmt(r['last'])}</td><td>{status_chip(r
['flags'])}</td>"
 378              f"<td class=notes>{escape(notes)}</td></tr><tr class=tl id=tl{i} hidden
><td colspan=5><ol>{tl}</ol></td></tr>"
 379          )
 380      link = start_link(request, quiz)
 381      msg = ""
 382      if imported is not None:
 383          msg = f"<p class=muted>Imported {imported} responses.</p>"
 384      if deleted:
 385          msg = "<p class=muted>All logs for this quiz were deleted.</p>"
 386      body = f"""<h1 style="font-size:22px;margin:0 0 16px">{escape(quiz['title'])}</
h1>
 387  <section><h2>Start link</h2><div class=row><input class=link id=link type=text read
only value="{escape(link)}">
 388  <button id=copy onclick="navigator.clipboard.writeText(document.getElementById('lin
k').value).then(()=>{{this.textContent='Copied';setTimeout(()=>this.textContent='Copy sta
rt link',2000)}})">Copy start link</button></div>
 389  <p class=muted>Send this link to the class. Students paste it into the Tapat extens
ion.</p></section>
 390  <section><h2>Students</h2><table><tr><th>Email</th><th>Events</th><th>Last event (M
anila)</th><th>Status</th><th>Notes</th></tr>
 391  {''.join(trs) or '<tr><td colspan=5 class=muted>No students yet.</td></tr>'}</table
>
 392  <p class=muted>Click a row to see its timeline.</p></section>
 393  <section><h2>Import responses</h2>{msg}
 394  <form method=post action="/admin/quiz/{code}/import" enctype=multipart/form-data cl
ass=row>
 395  <input type=file name=file accept=.csv required><button>Import responses</button></
form>
 396  <p class=muted>Upload the Form's responses CSV after the quiz. Anyone who submitted
without Tapat shows as No session.</p></section>
 397  <section><h2>Delete logs</h2><form method=post action="/admin/quiz/{code}/delete" c
lass=row>
 398  <input type=text name=confirm placeholder="Type DELETE" autocomplete=off oninput="t
his.form.querySelector('button').disabled=this.value!=='DELETE'">
 399  <button class=danger disabled>Delete logs</button></form>
 400  <p class=muted>Removes every event, session and imported response for this quiz. Do
this after grading.</p></section>
 401  <section><h2>Delete quiz</h2><form method=post action="/admin/quiz/{code}/delete-qu
iz" class=row>
 402  <input type=text name=confirm placeholder="Type DELETE" autocomplete=off oninput="t
his.form.querySelector('button').disabled=this.value!=='DELETE'">
 403  <button class=danger disabled>Delete quiz</button></form>
 404  <p class=muted>Removes this quiz, its start link and all of its logs. Students can
no longer start it.</p></section>"""
 405      return page(quiz["title"], body, nav=ADMIN_NAV)
 406
 407
 408  @app.post("/admin/quiz/{code}/import")
 409  async def admin_import(code: str, file: UploadFile = File(...), tapat_admin: str |
None = Cookie(None)):
 410      require_admin(tapat_admin)
 411      n = import_rows(code.upper(), (await file.read()).decode("utf-8-sig"))
 412      return RedirectResponse(f"/admin/quiz/{code.upper()}?imported={n}", status_code
```

```
=303)
 413
 414
 415  @app.post("/admin/quiz/{code}/delete")
 416  def admin_delete(code: str, confirm: str = Form(""), tapat_admin: str | None = Cook
ie(None)):
 417      require_admin(tapat_admin)
 418      if confirm != "DELETE":
 419          raise HTTPException(400, "Type DELETE to confirm")
 420      delete_logs(code.upper())
 421      return RedirectResponse(f"/admin/quiz/{code.upper()}?deleted=1", status_code=30
3)
 422
 423
 424  @app.post("/admin/quiz/{code}/delete-quiz")
 425  def admin_delete_quiz(code: str, confirm: str = Form(""), tapat_admin: str | None =
Cookie(None)):
 426      require_admin(tapat_admin)
 427      if confirm != "DELETE":
 428          raise HTTPException(400, "Type DELETE to confirm")
 429      code = code.upper()
 430      delete_logs(code)
 431      with db.connect() as c:
 432          c.execute("DELETE FROM quizzes WHERE code=?", (code,))
 433      return RedirectResponse("/admin", status_code=303)
```

server/db.py · Python · 75 lines

```python
"""Database access: Postgres when DATABASE_URL is set, SQLite otherwise.

Render's free disk is wiped on every deploy and restart, so the hosted
server keeps its data in a free Postgres database (Neon or Supabase).
Locally, a SQLite file is enough.
"""
import os
import sqlite3

DATABASE_URL = os.environ.get("DATABASE_URL", "").strip()
SQLITE_PATH = os.environ.get("TAPAT_DB", os.path.join(os.path.dirname(__file__), "tapat.db"))
IS_PG = DATABASE_URL.startswith(("postgres://", "postgresql://"))

SCHEMA = [
    """CREATE TABLE IF NOT EXISTS quizzes(
        code TEXT PRIMARY KEY, token TEXT NOT NULL, title TEXT NOT NULL, form_url TEXT NOT NULL,
        created DOUBLE PRECISION)""",
    """CREATE TABLE IF NOT EXISTS sessions(
        id TEXT PRIMARY KEY, token TEXT NOT NULL, quiz TEXT NOT NULL, email TEXT NOT NULL,
        created DOUBLE PRECISION)""",
    """CREATE TABLE IF NOT EXISTS events(
        id {pk}, quiz TEXT, session_id TEXT, seq INTEGER, student TEXT,
        type TEXT, detail TEXT, ts DOUBLE PRECISION, received DOUBLE PRECISION,
        UNIQUE (session_id, seq))""",
    """CREATE TABLE IF NOT EXISTS responses(
        quiz TEXT, email TEXT, submitted_at TEXT, PRIMARY KEY (quiz, email))""",
    "CREATE INDEX IF NOT EXISTS events_quiz ON events(quiz)",
    "CREATE INDEX IF NOT EXISTS events_session ON events(session_id)",
]


class Conn:
    """Small wrapper so the app writes one SQL dialect (`?` placeholders)."""

    def __init__(self):
        if IS_PG:
            import psycopg
            from psycopg.rows import dict_row
            self.raw = psycopg.connect(DATABASE_URL, row_factory=dict_row)
        else:
            self.raw = sqlite3.connect(SQLITE_PATH)
            self.raw.row_factory = sqlite3.Row

    def execute(self, sql, params=()):
        if IS_PG:
            sql = sql.replace("?", "%s")
        cur = self.raw.cursor()
        cur.execute(sql, params)
        return cur

    def executemany(self, sql, rows):
        if IS_PG:
            sql = sql.replace("?", "%s")
        self.raw.cursor().executemany(sql, rows)

    def __enter__(self):
        return self
```

```
58
59      def __exit__(self, exc, *_):
60          if exc:
61              self.raw.rollback()
62          else:
63              self.raw.commit()
64          self.raw.close()
65
66
67  def connect() -> Conn:
68      return Conn()
69
70
71  def init():
72      pk = "SERIAL PRIMARY KEY" if IS_PG else "INTEGER PRIMARY KEY"
73      with connect() as c:
74          for stmt in SCHEMA:
75              c.execute(stmt.format(pk=pk))
```

server/auth.py · Python · 60 lines

```python
"""Admin login: one password from ADMIN_PASSWORD, kept in a signed cookie."""
import hashlib
import hmac
import os
import secrets
import time

COOKIE = "tapat_admin"
MAX_AGE = 12 * 3600


def admin_password() -> str:
    # Set only in the environment (Render dashboard or run.sh), never in the repo.
    return os.environ.get("ADMIN_PASSWORD", "")


def _secret() -> bytes:
    key = os.environ.get("SECRET_KEY") or ("tapat:" + admin_password())
    return key.encode()


def check_password(given: str) -> bool:
    pw = admin_password()
    return bool(pw) and hmac.compare_digest(given.encode(), pw.encode())


def make_cookie() -> str:
    expires = str(int(time.time()) + MAX_AGE)
    nonce = secrets.token_hex(8)
    payload = f"{expires}.{nonce}"
    sig = hmac.new(_secret(), payload.encode(), hashlib.sha256).hexdigest()
    return f"{payload}.{sig}"


def valid_cookie(value: str | None) -> bool:
    if not value or value.count(".") != 2:
        return False
    expires, nonce, sig = value.split(".")
    good = hmac.new(_secret(), f"{expires}.{nonce}".encode(), hashlib.sha256).hexdig
est()
    return hmac.compare_digest(sig, good) and expires.isdigit() and int(expires) > t
ime.time()


# Login rate limit: 5 failed tries per address per 10 minutes.
FAIL_LIMIT = 5
FAIL_WINDOW = 600
_fails: dict[str, list[float]] = {}


def too_many_tries(ip: str) -> bool:
    now = time.time()
    _fails[ip] = [t for t in _fails.get(ip, []) if now - t < FAIL_WINDOW]
    return len(_fails[ip]) >= FAIL_LIMIT


def record_fail(ip: str):
    _fails.setdefault(ip, []).append(time.time())


```

```
59  def clear_fails(ip: str):
60      _fails.pop(ip, None)
```

server/seed.py · Python · 65 lines

```python
"""Create a demo quiz (code DEMO01) with three sample students."""
import os
import time

import db
from app import import_rows

QUIZ = "DEMO01"
FORM = "https://docs.google.com/forms/d/e/1FAIpQLSdemo/viewform"


def main():
    db.init()
    with db.connect() as c:
        for t in ("events", "responses", "sessions"):
            c.execute(f"DELETE FROM {t} WHERE quiz=?", (QUIZ,))
        c.execute("DELETE FROM quizzes WHERE code=?", (QUIZ,))
        c.execute("INSERT INTO quizzes(code, token, title, form_url, created) VALUES (?,?,?,?,?)",
                  (QUIZ, "demo-token", "INTE301 Quiz 2", FORM, time.time() - 3600))
        t0 = time.time() - 1200
        rows = []
        a, b = "student.a@school.edu", "student.b@school.edu"
        for sid, email in (("s-a", a), ("s-b", b)):
            c.execute("INSERT INTO sessions(id, token, quiz, email, created) VALUES (?,?,?,?,?)",
                      (sid, "seed-" + sid, QUIZ, email, t0))
        for i in range(0, 600, 10):  # Student A: clean run
            rows.append((QUIZ, "s-a", a, "heartbeat", "", t0 + i))
        rows.append((QUIZ, "s-a", a, "submitted", "formResponse", t0 + 600))
        t1 = t0 + 30  # Student B: left the exam, ChatGPT blocked, Word opened, silent gap
        for i in list(range(0, 200, 10)) + list(range(260, 700, 10)):
            rows.append((QUIZ, "s-b", b, "heartbeat", "", t1 + i))
        rows += [
            (QUIZ, "s-b", b, "focus_lost", "Chrome lost focus", t1 + 95),
            (QUIZ, "s-b", b, "tab_switch", "new tab", t1 + 120),
            (QUIZ, "s-b", b, "blocked_url", "chatgpt.com", t1 + 124),
            (QUIZ, "s-b", b, "app_opened", "Microsoft Word", t1 + 300),
            (QUIZ, "s-b", b, "tab_switch", "returned to exam", t1 + 330),
            (QUIZ, "s-b", b, "submitted", "formResponse", t1 + 700),
        ]
        # Student D: wifi dropped for about 90 seconds; the extension queued a
        # focus change and the heartbeats, then sent them when back online.
        d = "student.d@school.edu"
        c.execute("INSERT INTO sessions(id, token, quiz, email, created) VALUES (?,?,?,?,?)",
                  ("s-d", "seed-s-d", QUIZ, d, t0))
        t2 = t0 + 60
        back_online = t2 + 345
        for i in list(range(0, 250, 10)) + [340] + list(range(350, 640, 10)):
            rows.append((QUIZ, "s-d", d, "heartbeat", "4 heartbeats while offline" if i == 340 else "", t2 + i))
        rows.append((QUIZ, "s-d", d, "focus_lost", "Chrome lost focus", t2 + 300))
        rows.append((QUIZ, "s-d", d, "submitted", "formResponse", t2 + 640))

        def received(sid, ts):
            # Queued while offline -> received when the wifi came back.
            return back_online + 2 if sid == "s-d" and t2 + 250 < ts <= back_online else ts + 0.3
```

```
55
56          rows = [(q, sid, n, who, t, det, ts, received(sid, ts)) for n, (q, sid, who,
t, det, ts) in enumerate(rows)]
57          c.executemany("INSERT INTO events(quiz, session_id, seq, student, type, deta
il, ts, received) "
58                        "VALUES (?,?,?,?,?,?,?,?)", rows)
59      # Student C is in the Form CSV but never ran Tapat (took it in incognito).
60      import_rows(QUIZ, open(os.path.join(os.path.dirname(__file__), "sample_response
s.csv")).read())
61      print(f"seeded quiz {QUIZ}")
62
63
64  if __name__ == "__main__":
65      main()
```

server/run.sh · Bash · 13 lines

```bash
#!/usr/bin/env bash
# Start Tapat locally on http://localhost:8000 (add --seed for demo data).
# The admin password comes from ADMIN_PASSWORD. If it is not set, a random
# one is made for this run and printed below.
set -e
cd "$(dirname "$0")"
[ -d .venv ] || { python3 -m venv .venv && .venv/bin/pip install -q -r requirements.txt; }
if [ -z "$ADMIN_PASSWORD" ]; then
  export ADMIN_PASSWORD="$(.venv/bin/python -c 'import secrets; print(secrets.token_urlsafe(9))')"
  echo "Admin password for this run: $ADMIN_PASSWORD"
fi
[ "$1" = "--seed" ] && .venv/bin/python seed.py
exec .venv/bin/uvicorn app:app --host 127.0.0.1 --port "${PORT:-8000}"
```

server/requirements.txt · Text · 6 lines

```
fastapi
uvicorn
python-multipart
psycopg[binary]
pytest
httpx
```

server/sample_responses.csv · CSV · 5 lines

```csv
Timestamp,Email Address,Score
10/12/2026 14:10:02,student.a@school.edu,18 / 20
10/12/2026 14:12:40,student.b@school.edu,17 / 20
10/12/2026 14:09:11,student.c@school.edu,20 / 20
10/12/2026 14:13:05,student.d@school.edu,16 / 20
```

server/tests/test_app.py · Python · 207 lines

```python
import os
import sys
import time

sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
os.environ["TAPAT_DB"] = "/tmp/tapat_test.db"
os.environ["ADMIN_PASSWORD"] = "test-only-password"
os.environ.pop("DATABASE_URL", None)
if os.path.exists("/tmp/tapat_test.db"):
    os.remove("/tmp/tapat_test.db")

from fastapi.testclient import TestClient  # noqa: E402

import app as tapat  # noqa: E402
import auth  # noqa: E402

c = TestClient(tapat.app)
QUIZ = tapat.create_quiz("Test quiz", "https://docs.google.com/forms/d/e/x/viewform")
TOKEN = tapat.get_quiz(QUIZ)["token"]


def start(email="a@x.edu", token=None):
    return c.post("/api/start", json={"code": QUIZ, "token": token or TOKEN, "email": email})


SEQ = {}


def ev(sess, t, detail="", ts=None, seq=None):
    if seq is None:
        seq = SEQ[sess["session_id"]] = SEQ.get(sess["session_id"], 0) + 1
    return c.post("/events", json={"session_id": sess["session_id"], "session_token": sess["session_token"],
                                   "type": t, "detail": detail, "ts": ts, "seq": seq})


A = start().json()


def login(client):
    return client.post("/admin/login", data={"password": "test-only-password"}, follow_redirects=False)


def test_start_needs_quiz_token():
    assert start(token="wrong").status_code == 403
    assert A["form_url"].endswith("/viewform") and A["session_token"]


def test_event_and_summary():
    assert ev(A, "blocked_url", "chatgpt.com").json()["ok"]
    row = [r for r in tapat.summarize(QUIZ) if r["email"] == "a@x.edu"][0]
    assert "Blocked site" in row["flags"]


def test_cannot_fake_events_for_classmate():
    forged = dict(A, session_token="guess")
```

```
    assert ev(forged, "heartbeat").status_code == 403
    # The email in the body is ignored; the session's own email is used.
    r = c.post("/events", json={"session_id": A["session_id"], "session_token": A
["session_token"],
                               "student": "victim@x.edu", "type": "tab_switch", "s
eq": 999})
    assert r.status_code == 200
    assert "victim@x.edu" not in [r["email"] for r in tapat.summarize(QUIZ)]


def test_rejects_unknown_type():
    assert ev(A, "bogus").status_code == 400


def test_start_page_needs_token():
    assert c.get(f"/q/{QUIZ}?t={TOKEN}").status_code == 200
    assert c.get(f"/q/{QUIZ}?t=nope").status_code == 404


def test_session_closes_on_submit():
    s = start("c@x.edu").json()
    assert c.get("/sessions/unknown").json()["state"] == "unknown"
    assert c.get(f"/sessions/{s['session_id']}").json()["state"] == "open"
    ev(s, "submitted")
    assert c.get(f"/sessions/{s['session_id']}").json()["state"] == "closed"


def test_monitor_silent():
    s = start("b@x.edu").json()
    now = time.time()
    ev(s, "heartbeat", ts=now - 100)
    ev(s, "heartbeat", ts=now - 60)
    row = [r for r in tapat.summarize(QUIZ) if r["email"] == "b@x.edu"][0]
    assert "Monitor silent" in row["flags"]


def test_admin_requires_login():
    fresh = TestClient(tapat.app)
    r = fresh.get("/admin", follow_redirects=False)
    assert r.status_code == 303 and r.headers["location"] == "/admin/login"
    assert fresh.get(f"/admin/quiz/{QUIZ}", follow_redirects=False).status_code ==
303
    assert fresh.post(f"/admin/quiz/{QUIZ}/delete", data={"confirm": "DELETE"}, fol
low_redirects=False).status_code == 303


def test_forged_cookie_rejected():
    assert not auth.valid_cookie("9999999999.abc.deadbeef")
    fresh = TestClient(tapat.app)
    fresh.cookies.set(auth.COOKIE, "9999999999.abc.deadbeef")
    assert fresh.get("/admin", follow_redirects=False).status_code == 303


def test_login_rate_limited():
    auth._fails.clear()
    fresh = TestClient(tapat.app)
    for _ in range(auth.FAIL_LIMIT):
        r = fresh.post("/admin/login", data={"password": "nope"}, follow_redirects=
False)
        assert "error=1" in r.headers["location"]
    r = login(fresh)  # even the right password is refused while locked out
    assert "error=rate" in r.headers["location"]
    auth._fails.clear()
```

```
114
115
116 def test_no_password_means_no_login(monkeypatch):
117     monkeypatch.setenv("ADMIN_PASSWORD", "")
118     assert not auth.check_password("")
119
120
121 def test_admin_create_quiz_and_page():
122     auth._fails.clear()
123     a = TestClient(tapat.app)
124     assert login(a).headers["location"] == "/admin"
125     r = a.post("/admin/quiz", data={"title": "Quiz 9", "form_url": "https://docs.go
ogle.com/forms/d/e/y/viewform"},
126                follow_redirects=False)
127     code = r.headers["location"].rsplit("/", 1)[1]
128     html = a.get(f"/admin/quiz/{code}").text
129     assert f"/q/{code}?t={tapat.get_quiz(code)['token']}" in html
130     for label in ("Copy start link", "Import responses", "Delete logs", "Type DELET
E"):
131         assert label in html
132     bad = a.post("/admin/quiz", data={"title": "x", "form_url": "https://example.co
m"}, follow_redirects=False)
133     assert bad.status_code == 400
134
135
136 def test_csv_no_session_and_delete_logs():
137     auth._fails.clear()
138     a = TestClient(tapat.app)
139     login(a)
140     csv = "Timestamp,Email Address\n1,a@x.edu\n2,ghost@x.edu\n"
141     a.post(f"/admin/quiz/{QUIZ}/import", files={"file": ("r.csv", csv, "text/cs
v")}, follow_redirects=False)
142     rows = {r["email"]: r for r in tapat.summarize(QUIZ)}
143     assert rows["ghost@x.edu"]["flags"] == ["No session"]
144     assert "No session" not in rows["a@x.edu"]["flags"]
145     assert a.post(f"/admin/quiz/{QUIZ}/delete", data={"confirm": "nope"}).status_co
de == 400
146     a.post(f"/admin/quiz/{QUIZ}/delete", data={"confirm": "DELETE"}, follow_redirec
ts=False)
147     assert tapat.summarize(QUIZ) == []
148     assert ev(A, "heartbeat").status_code == 403  # sessions were deleted too
149
150
151 def test_resend_is_deduped_by_session_and_seq():
152     s = start("dup@x.edu").json()
153     assert ev(s, "tab_switch", "new tab", seq=1).json()["duplicate"] is False
154     assert ev(s, "tab_switch", "new tab", seq=1).json()["duplicate"] is True
155     row = [r for r in tapat.summarize(QUIZ) if r["email"] == "dup@x.edu"][0]
156     assert row["counts"]["tab_switch"] == 1
157
158
159 def test_queued_events_keep_client_time_and_order():
160     s = start("order@x.edu").json()
161     t = time.time() - 300
162     # Sent out of order (as a retry might), each with its own client time.
163     ev(s, "submitted", "formResponse", ts=t + 30, seq=3)
164     ev(s, "focus_lost", "Chrome lost focus", ts=t + 10, seq=1)
165     ev(s, "blocked_url", "chatgpt.com", ts=t + 20, seq=2)
166     row = [r for r in tapat.summarize(QUIZ) if r["email"] == "order@x.edu"][0]
167     assert [k for _, k, _ in row["timeline"]] == ["focus_lost", "blocked_url", "sub
mitted"]
168     with tapat.db.connect() as conn:
169         got = conn.execute("SELECT ts, received FROM events WHERE session_id=? AND
```

```
seq=1", (s["session_id"],)).fetchone()
    assert abs(got["ts"] - (t + 10)) < 0.01 and got["received"] > got["ts"] + 200


def test_offline_gap_synced_later_is_not_monitor_silent():
    s = start("offline@x.edu").json()
    t = time.time() - 600
    for i in range(0, 60, 10):
        ev(s, "heartbeat", ts=t + i)
    # Wifi drops for 90s. The queued summary heartbeat arrives late, then live beat
s resume.
    ev(s, "heartbeat", "9 heartbeats while offline", ts=t + 150)
    for i in range(160, 200, 10):
        ev(s, "heartbeat", ts=t + i)
    ev(s, "submitted", ts=t + 200)
    row = [r for r in tapat.summarize(QUIZ) if r["email"] == "offline@x.edu"][0]
    assert "Offline gap, synced later" in row["flags"]
    assert "Monitor silent" not in row["flags"]


def test_delete_quiz_removes_everything():
    auth._fails.clear()
    a = TestClient(tapat.app)
    login(a)
    code = tapat.create_quiz("Doomed", "https://docs.google.com/forms/d/e/z/viewfor
m")
    tok = tapat.get_quiz(code)["token"]
    s = c.post("/api/start", json={"code": code, "token": tok, "email": "d@x.ed
u"}).json()
    ev(s, "heartbeat")
    a.post(f"/admin/quiz/{code}/import", files={"file": ("r.csv", "Email Address\nz
@x.edu\n", "text/csv")})
    assert a.post(f"/admin/quiz/{code}/delete-quiz", data={"confirm": "nope"}).stat
us_code == 400
    assert TestClient(tapat.app).post(f"/admin/quiz/{code}/delete-quiz", data={"con
firm": "DELETE"},
                                      follow_redirects=False).headers["location"] =
= "/admin/login"
    r = a.post(f"/admin/quiz/{code}/delete-quiz", data={"confirm": "DELETE"}, follo
w_redirects=False)
    assert r.status_code == 303 and r.headers["location"] == "/admin"
    assert tapat.get_quiz(code) is None
    with tapat.db.connect() as conn:
        for table in ("events", "sessions", "responses"):
            assert conn.execute(f"SELECT COUNT(*) AS n FROM {table} WHERE quiz=?",
(code,)).fetchone()["n"] == 0
    assert c.post("/api/start", json={"code": code, "token": tok, "email": "d@x.ed
u"}).status_code == 403
    assert "Doomed" not in a.get("/admin").text
```

render.yaml · YAML · 19 lines

```yaml
# Render Blueprint for Tapat (free plan). See docs/deploy.md.
services:
  - type: web
    name: tapat
    runtime: python
    plan: free
    rootDir: server
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app:app --host 0.0.0.0 --port $PORT --proxy-headers --forwarded-allow-ips '*'
    healthCheckPath: /health
    envVars:
      - key: PYTHON_VERSION
        value: 3.12.7
      - key: DATABASE_URL
        sync: false
      - key: ADMIN_PASSWORD
        sync: false
      - key: SECRET_KEY
        generateValue: true
```

Appendix B2. Source code: Chrome extension

11 files.

extension/manifest.json · JSON · 36 lines

```json
{
  "manifest_version": 3,
  "name": "Tapat",
  "version": "1.1.0",
  "description": "Exam monitor for Google Forms quizzes: blocks a listed set of sites and logs leaving the exam.",
  "permissions": [
    "declarativeNetRequest",
    "webNavigation",
    "tabs",
    "storage",
    "alarms"
  ],
  "host_permissions": [
    "https://docs.google.com/*",
    "https://tapat-xrg5.onrender.com/*"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup.html",
    "default_title": "Tapat",
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "48": "icons/icon-48.png",
      "128": "icons/icon-128.png"
    }
  },
  "icons": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  }
}
```

extension/config.js · JavaScript · 28 lines

```
// Shared settings for the Tapat extension.
// Default hosted server. Change it after deploying (see docs/deploy.md);
// a start link from any server also works, since it carries its own address.
const DEFAULT_SERVER = "https://tapat-xrg5.onrender.com";
const HEARTBEAT_SECONDS = 10;

// Sites blocked while an exam session is active. These are RE2 regexes on the
// full URL. Google Forms (docs.google.com/forms/...) must never match.
const BLOCK_LIST = [
  { label: "chatgpt.com", regex: "^https?://([a-z0-9-]+\\.)*(chatgpt\\.com|chat\\.openai\\.com)/" },
  { label: "gemini.google.com", regex: "^https?://gemini\\.google\\.com/" },
  { label: "claude.ai", regex: "^https?://([a-z0-9-]+\\.)*claude\\.ai/" },
  { label: "Word Online / office.com", regex: "^https?://([a-z0-9-]+\\.)*(office\\.com|officeapps\\.live\\.com)/" },
  { label: "Google Docs", regex: "^https?://docs\\.google\\.com/document/" },
  { label: "PDF file", regex: "^[^?#]*\\.pdf([?#].*)?$" }
];

// Start links look like https://<server>/q/<CODE>?t=<quiz token>
function parseStartLink(text) {
  let url;
  try { url = new URL(text.trim()); } catch (e) { return null; }
  const m = url.pathname.match(/^\/q\/([A-Za-z0-9]+)\/?$/);
  const token = url.searchParams.get("t");
  if (!m || !token) return null;
  return { server: url.origin, code: m[1].toUpperCase(), token };
}

if (typeof module !== "undefined") module.exports = { BLOCK_LIST, parseStartLink, DEFAULT_SERVER };
```

extension/background.js · JavaScript · 222 lines

```
// Tapat service worker: session start, blocking, focus logging, submit
// detection, heartbeat, and an event queue that survives a sleeping server.
importScripts("config.js");

const store = chrome.storage.local;

async function getSession() {
  const { session } = await store.get("session");
  return session || null;
}

// ---------- event queue ----------
// Every event is written to chrome.storage.local first, with its own client
// time and a per-session sequence number, then sent in order. If the student
// has no wifi, or the free server is still waking up, events stay queued and
// are retried with backoff (on a timer, on a chrome alarm, and when the
// browser comes back online). The server ignores a (session, seq) it already
// has, so a resend can never double count.
let flushing = false;
let retryTimer = null;
// Queue writes go through one chain so two events can't overwrite each other.
let lock = Promise.resolve();
function locked(fn) {
  const run = lock.then(fn);
  lock = run.catch(() => {});
  return run;
}

function enqueue(type, detail = "") {
  return locked(() => addToQueue(type, detail));
}

async function addToQueue(type, detail) {
  const s = await getSession();
  if (!s || !s.active) return;
  const { queue = [], backoff = 0, seq = 0 } = await store.get(["queue", "backoff",
"seq"]);
  const now = Date.now() / 1000;
  let nextSeq = seq;
  const last = queue[queue.length - 1];
  const offline = backoff > 0 || !navigator.onLine;
  if (type === "heartbeat" && offline && queue.length > 1 && last.body.type === "he
artbeat" && last.body.session_id === s.id) {
    // Offline: fold heartbeats into one summary instead of piling them up.
    last.beats = (last.beats || 1) + 1;
    last.body.ts = now;
    last.body.detail = `${last.beats} heartbeats while offline`;
  } else {
    nextSeq = seq + 1;
    queue.push({ server: s.server, body: {
      session_id: s.id, session_token: s.token, seq: nextSeq, student: s.email, typ
e, detail, ts: now
    } });
  }
  await store.set({ queue, seq: nextSeq });
  flush();
}

async function flush() {
  if (flushing) return;
```

```
  58    flushing = true;
  59    try {
  60      for (;;) {
  61        const { queue = [], backoff = 0 } = await store.get(["queue", "backoff"]);
  62        if (!queue.length) { await store.set({ backoff: 0 }); chrome.alarms.clear("fl
ush"); break; }
  63        const item = queue[0];
  64        let res = null;
  65        if (navigator.onLine) {
  66          try {
  67            res = await fetch(`${item.server}/events`, {
  68              method: "POST", headers: { "Content-Type": "application/json" }, body:
JSON.stringify(item.body)
  69            });
  70          } catch (e) { res = null; }
  71        }
  72        if (res && (res.ok || (res.status >= 400 && res.status < 500))) {
  73          // Sent (or a duplicate), or rejected for good (bad token): drop it, keep o
rder.
  74          await locked(async () => {
  75            const { queue: q2 = [] } = await store.get("queue");
  76            q2.shift();
  77            await store.set({ queue: q2, backoff: 0 });
  78          });
  79          continue;
  80        }
  81        // Offline, asleep or unreachable: wait 2, 4, 8 ... up to 60 seconds.
  82        const next = Math.min(backoff ? backoff * 2 : 2, 60);
  83        await store.set({ backoff: next });
  84        clearTimeout(retryTimer);
  85        retryTimer = setTimeout(flush, next * 1000);
  86        // The alarm survives the service worker being shut down.
  87        chrome.alarms.create("flush", { periodInMinutes: 0.5 });
  88        break;
  89      }
  90    } finally {
  91      flushing = false;
  92    }
  93  }
  94
  95  self.addEventListener("online", () => { store.set({ backoff: 0 }).then(flush); });
  96
  97  // ---------- server calls with wake-up retries ----------
  98  async function fetchRetry(url, options, totalSeconds = 120) {
  99    const until = Date.now() + totalSeconds * 1000;
 100    let wait = 2;
 101    for (;;) {
 102      try {
 103        const res = await fetch(url, options);
 104        if (res.status < 500) return res;
 105      } catch (e) { /* asleep or offline */ }
 106      if (Date.now() > until) throw new Error("Server did not answer");
 107      await new Promise(r => setTimeout(r, wait * 1000));
 108      wait = Math.min(wait * 2, 15);
 109    }
 110  }
 111
 112  // ---------- blocking ----------
 113  async function enableBlocking() {
 114    const rules = BLOCK_LIST.map((b, i) => ({
 115      id: i + 1, priority: 1, action: { type: "block" },
 116      condition: { regexFilter: b.regex, resourceTypes: ["main_frame", "sub_frame"] }
 117    }));
```

```
  const old = await chrome.declarativeNetRequest.getDynamicRules();
  await chrome.declarativeNetRequest.updateDynamicRules({ removeRuleIds: old.map(r => r.id), addRules: rules });
}

async function disableBlocking() {
  const old = await chrome.declarativeNetRequest.getDynamicRules();
  await chrome.declarativeNetRequest.updateDynamicRules({ removeRuleIds: old.map(r => r.id) });
}

// A blocked page fails with ERR_BLOCKED_BY_CLIENT; log it and show our page.
chrome.webNavigation.onErrorOccurred.addListener(async (d) => {
  const s = await getSession();
  if (!s || !s.active || d.error !== "net::ERR_BLOCKED_BY_CLIENT") return;
  let shown = d.url;
  try { const u = new URL(d.url); shown = u.hostname + u.pathname.slice(0, 40); } catch (e) {}
  enqueue("blocked_url", shown);
  if (d.frameId === 0) {
    chrome.tabs.update(d.tabId, { url: chrome.runtime.getURL("blocked.html") + "#" + encodeURIComponent(shown) });
  }
});

// ---------- session ----------
async function startSession({ email, link }) {
  const parsed = parseStartLink(link);
  if (!parsed) return { ok: false, error: "That start link is not valid." };
  let res;
  try {
    res = await fetchRetry(`${parsed.server}/api/start`, {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ code: parsed.code, token: parsed.token, email })
    });
  } catch (e) {
    return { ok: false, error: "The Tapat server did not answer. Check your internet and try again." };
  }
  const data = await res.json();
  if (!res.ok) return { ok: false, error: data.error || "Could not start the quiz." };
  const session = {
    id: data.session_id, token: data.session_token, email: email.trim().toLowerCase(), server: parsed.server,
    quiz: data.quiz, title: data.title, formUrl: data.form_url, active: true
  };
  await store.set({ session, seq: 0, examTabId: null });
  await enableBlocking();
  chrome.alarms.create("heartbeat", { periodInMinutes: HEARTBEAT_SECONDS / 60 });
  enqueue("heartbeat", "session started");
  return { ok: true, session };
}

async function endSession(reason) {
  await enqueue("submitted", reason);
  await locked(async () => {
    const s = await getSession();
    if (s) await store.set({ session: { ...s, active: false } });
  });
  chrome.alarms.clear("heartbeat");
  await disableBlocking();
}
```

```
174
175  chrome.runtime.onMessage.addListener((msg, sender, reply) => {
176    if (msg.kind === "start") { startSession(msg).then(reply); return true; }
177    if (msg.kind === "examTab") {
178      store.set({ examTabId: sender.tab.id });
179    }
180    if (msg.kind === "status") { store.get("examTabId").then(({ examTabId }) => reply
({ examTabId })); return true; }
181    if (msg.kind === "queue") {
182      store.get(["queue", "backoff"]).then(({ queue = [], backoff = 0 }) =>
183        reply({ queued: queue.length, offline: !navigator.onLine || backoff > 0 }));
184      return true;
185    }
186  });
187
188  chrome.alarms.onAlarm.addListener(a => {
189    if (a.name === "heartbeat") enqueue("heartbeat");
190    if (a.name === "flush") flush();
191  });
192  chrome.runtime.onStartup.addListener(flush);
193
194  // Chrome itself lost focus (another app or window in front).
195  chrome.windows.onFocusChanged.addListener((windowId) => {
196    if (windowId === chrome.windows.WINDOW_ID_NONE) enqueue("focus_lost", "Chrome los
t focus");
197  });
198
199  // Leaving the exam tab.
200  chrome.tabs.onActivated.addListener(async ({ tabId }) => {
201    const s = await getSession();
202    const { examTabId = null } = await store.get("examTabId");
203    if (s && s.active && examTabId !== null && tabId !== examTabId) {
204      const tab = await chrome.tabs.get(tabId).catch(() => null);
205      let where = "other tab";
206      try {
207        if (tab && tab.url) {
208          const u = new URL(tab.url);
209          where = u.protocol === "chrome-extension:" ? "Tapat page" : (u.hostname ||
where);
210        }
211      } catch (e) {}
212      enqueue("tab_switch", where);
213    }
214  });
215  chrome.tabs.onCreated.addListener(() => enqueue("tab_switch", "new tab"));
216
217  // The embedded Form frame lands on .../formResponse after submit.
218  chrome.webNavigation.onCompleted.addListener(async (d) => {
219    const s = await getSession();
220    const { examTabId = null } = await store.get("examTabId");
221    if (s && s.active && d.tabId === examTabId && d.url.includes("formResponse")) end
Session("formResponse");
222  }, { url: [{ hostEquals: "docs.google.com" }] });
```

extension/popup.html · HTML · 18 lines

```
<!doctype html><html><head><meta charset="utf-8"><link rel="stylesheet" href="style.css"></head>
 <body class="popup">
   <h1><img src="icons/icon-32.png" alt="" style="height:24px;vertical-align:-4px;margin-right:6px">Tapat</h1>
   <p class="muted">Start your quiz here.</p>
   <label>Start link<input id="link" type="url" placeholder="Paste the link from your professor"></label>
   <label>School email<input id="email" type="email" placeholder="you@school.edu"></label>
   <button id="go">Open exam</button>
   <p id="queue" class="state" hidden></p>
   <p id="state" class="state" hidden></p>
   <p id="err" class="err"></p>
   <details class="adv"><summary>Server</summary>
     <label>Server address<input id="server" type="url"></label>
     <button id="save" class="secondary">Save</button>
     <p class="muted">Used when the start link is only a code. A full start link carries its own server.</p>
   </details>
   <script src="config.js"></script>
   <script src="popup.js"></script>
 </body></html>
```

extension/popup.js · JavaScript · 58 lines

```
const $ = (id) => document.getElementById(id);

chrome.storage.local.get(["server", "lastEmail"]).then(({ server, lastEmail }) => {
  $("server").value = server || DEFAULT_SERVER;
  if (lastEmail) $("email").value = lastEmail;
});

$("save").onclick = async () => {
  await chrome.storage.local.set({ server: $("server").value.trim().replace(/\/$/, "") });
  $("save").textContent = "Saved";
};

// The free server sleeps when idle and can take about a minute to wake.
async function waitForServer(server) {
  const until = Date.now() + 120000;
  let first = true;
  for (;;) {
    try {
      const res = await fetch(`${server}/health`, { cache: "no-store" });
      if (res.ok) return true;
    } catch (e) { /* still waking */ }
    if (first) { $("state").hidden = false; $("state").textContent = "Server waking up… this can take a minute."; first = false; }
    if (Date.now() > until) return false;
    await new Promise(r => setTimeout(r, 3000));
  }
}

$("go").onclick = async () => {
  const email = $("email").value.trim();
  let link = $("link").value.trim();
  $("err").textContent = "";
  // A bare code like Q7K2PM?t=... is accepted with the saved server.
  if (link && !/^https?:/.test(link)) link = `${$("server").value.trim().replace(/\/$/, "")}/q/${link.replace(/^\/?q\//, "")}`;
  const parsed = parseStartLink(link);
  if (!parsed) return ($("err").textContent = "Paste the start link from your professor.");
  if (!/^\S+@\S+\.\S+$/.test(email)) return ($("err").textContent = "Enter your school email.");
  $("go").disabled = true;
  const up = await waitForServer(parsed.server);
  $("go").disabled = false;
  $("state").hidden = true;
  if (!up) return ($("err").textContent = "The Tapat server did not answer. Try again in a minute.");
  await chrome.storage.local.set({ lastEmail: email });
  const url = chrome.runtime.getURL(`exam.html?email=${encodeURIComponent(email)}&link=${encodeURIComponent(link)}`);
  chrome.tabs.create({ url });
  window.close();
};

// Queue status: shown while events are waiting to be sent.
async function showQueue() {
  const q = await chrome.runtime.sendMessage({ kind: "queue" });
  const el = $("queue");
  if (q && q.queued > 0 && q.offline) {
    el.hidden = false;
```

```
54      el.textContent = `Offline, ${q.queued} events queued. They will send when you ar
e back online.`;
55    } else el.hidden = true;
56  }
57  showQueue();
58  setInterval(showQueue, 2000);
```

extension/exam.html · HTML · 14 lines

```
<!doctype html><html><head><meta charset="utf-8"><title>Tapat exam</title><link rel="stylesheet" href="style.css"></head>
<body>
  <header class="top"><b><img src="icons/icon-32.png" alt="" style="height:18px;vertical-align:-4px;margin-right:6px">Tapat</b><span><span id="who"></span> <button id="watch" class="secondary" hidden>Copy watcher command</button></span></header>
  <section id="consent" class="card">
    <h2>Before you start</h2>
    <p class="big">This logs tab switches and apps opened, nothing else.</p>
    <p>While the quiz is open, a listed set of sites is blocked. Your professor sees a timeline of when you left the exam. Logs are deleted after grading.</p>
    <button id="agree">I understand, start the quiz</button>
    <p id="state" class="state" hidden></p>
    <p id="err" class="err"></p>
  </section>
  <iframe id="form" hidden></iframe>
  <script src="exam.js"></script>
</body></html>
```

extension/exam.js · JavaScript · 39 lines

```
const params = new URLSearchParams(location.search);
const email = params.get("email");
const link = params.get("link") || "";
document.getElementById("who").textContent = email;

function embedUrl(u) {
  const url = new URL(u);
  url.searchParams.set("embedded", "true");
  return url.toString();
}

document.getElementById("agree").onclick = async (e) => {
  e.target.disabled = true;
  const state = document.getElementById("state");
  state.hidden = false;
  state.textContent = "Starting… if the server is waking up, this can take a minut
e.";
  const res = await chrome.runtime.sendMessage({ kind: "start", email, link });
  state.hidden = true;
  if (!res || !res.ok) {
    e.target.disabled = false;
    document.getElementById("err").textContent = (res && res.error) || "Could not st
art the quiz.";
    return;
  }
  chrome.runtime.sendMessage({ kind: "examTab" });
  document.title = res.session.title;
  document.getElementById("who").textContent = `${email} · ${res.session.title}`;
  const w = document.getElementById("watch");
  w.hidden = false;
  w.onclick = async () => {
    const s = res.session;
    await navigator.clipboard.writeText(
      `python watcher.py --email ${s.email} --session ${s.id} --token ${s.token} --s
erver ${s.server}`);
    w.textContent = "Copied";
  };
  document.getElementById("consent").hidden = true;
  const frame = document.getElementById("form");
  frame.src = embedUrl(res.session.formUrl);
  frame.hidden = false;
};
```

extension/blocked.html · HTML · 10 lines

```
<!doctype html><html><head><meta charset="utf-8"><title>Blocked by Tapat</title><link rel="stylesheet" href="style.css"></head>
<body class="blocked">
  <div class="card">
    <h1>Blocked during your quiz</h1>
    <p id="site" class="mono"></p>
    <p>This site is on your professor's block list while the exam is open. The attempt was logged.</p>
    <button id="back">Back to the exam</button>
  </div>
  <script src="blocked.js"></script>
</body></html>
```

extension/blocked.js · JavaScript · 7 lines

```
// Shown after the service worker sees a blocked navigation (it already logged it).
document.getElementById("site").textContent = decodeURIComponent(location.hash.slice(1)) || "blocked site";
document.getElementById("back").onclick = async () => {
  const s = await chrome.runtime.sendMessage({ kind: "status" });
  if (s && s.examTabId) chrome.tabs.update(s.examTabId, { active: true });
  window.close();
};
```

extension/style.css · CSS · 18 lines

```css
body{font-family:Inter,Segoe UI,Helvetica,Arial,sans-serif;margin:0;color:#111;background:#f5f7fb}
.popup{width:300px;padding:16px;background:#fff}
.popup h1{color:#1F3A8A;margin:0 0 4px}
label{display:block;font-size:13px;margin:10px 0}
input{display:block;width:100%;box-sizing:border-box;padding:8px;margin-top:4px;border:1px solid #ccd;border-radius:6px}
button{background:#1F3A8A;color:#fff;border:0;border-radius:6px;padding:10px 16px;font-size:14px;cursor:pointer}
.muted{color:#666;font-size:13px}.err{color:#b91c1c;font-size:13px}
.top{background:#1F3A8A;color:#fff;padding:12px 24px;display:flex;justify-content:space-between}
.card{max-width:560px;margin:60px auto;background:#fff;padding:28px;border-radius:10px;box-shadow:0 2px 8px #0002}
.card h2,.card h1{color:#1F3A8A;margin-top:0}
.big{font-size:18px;font-weight:600}
iframe{width:100%;height:calc(100vh - 46px);border:0;display:block}
.blocked{background:#fff1f1}.blocked h1{color:#9b1c1c!important}
.mono{font-family:Consolas,monospace;background:#f3f3f3;padding:6px 10px;border-radius:4px}
.state{background:#fff7e0;border-left:3px solid #e0a400;padding:6px 10px;font-size:13px}
.adv{margin-top:12px;font-size:13px}.adv summary{cursor:pointer;color:#555}
button.secondary{background:#e3e8f5;color:#1F3A8A;padding:6px 12px;font-size:13px}
button:disabled{opacity:.5}
```

extension/tests/blocklist.test.js · JavaScript · 37 lines

```
// Run with: node --test extension/tests
// The block list must catch Google Docs but never the Google Form itself.
const test = require("node:test");
const assert = require("node:assert");
const { BLOCK_LIST, parseStartLink } = require("../config.js");

const blocked = (url) => BLOCK_LIST.some((b) => new RegExp(b.regex).test(url));

test("Google Docs documents are blocked", () => {
  assert.ok(blocked("https://docs.google.com/document/d/1abc/edit"));
});

test("Google Forms are never blocked", () => {
  for (const url of [
    "https://docs.google.com/forms/d/e/1FAIpQLSf/viewform?embedded=true",
    "https://docs.google.com/forms/d/e/1FAIpQLSf/formResponse",
    "https://docs.google.com/forms/d/e/1FAIpQLSf/viewform?usp=pp_url&entry.1=report.pdf",
  ]) assert.ok(!blocked(url), url);
});

test("listed sites are blocked", () => {
  for (const url of [
    "https://chatgpt.com/", "https://chat.openai.com/c/1", "https://gemini.google.com/app",
    "https://claude.ai/new", "https://www.office.com/launch/word", "https://example.com/notes/lesson1.pdf",
  ]) assert.ok(blocked(url), url);
});

test("unlisted sites stay open", () => {
  assert.ok(!blocked("https://www.google.com/search?q=api+gateway"));
});

test("start links parse", () => {
  assert.deepStrictEqual(parseStartLink("https://tapat-xrg5.onrender.com/q/q7k2pm?t=abc"),
    { server: "https://tapat-xrg5.onrender.com", code: "Q7K2PM", token: "abc" });
  assert.strictEqual(parseStartLink("https://tapat-xrg5.onrender.com/q/Q7K2PM"), null);
  assert.strictEqual(parseStartLink("not a link"), null);
});
```

Appendix B3. Source code: Desktop watcher

3 files.

watcher/watcher.py · Python · 90 lines

```python
"""Tapat desktop watcher: logs which watched apps come to the front.

Only app names are sent, never window titles. Exits when the exam
session is closed on the server (the student submitted the Form).
"""
import argparse
import json
import platform
import subprocess
import sys
import time
import urllib.request

DEFAULT_APPS = ["Microsoft Word", "WINWORD", "Notes", "Notepad", "Preview", "Acroba
t", "AcroRd32"]


def frontmost_app() -> str | None:
    system = platform.system()
    if system == "Darwin":
        out = subprocess.run(
            ["osascript", "-e", 'tell application "System Events" to get name of fir
st process whose frontmost is true'],
            capture_output=True, text=True,
        )
        return out.stdout.strip() or None
    if system == "Windows":
        import psutil
        import win32gui
        import win32process
        hwnd = win32gui.GetForegroundWindow()
        _, pid = win32process.GetWindowThreadProcessId(hwnd)
        try:
            return psutil.Process(pid).name().removesuffix(".exe")
        except psutil.Error:
            return None
    return None


def matches(app: str, watched: list[str]) -> str | None:
    for name in watched:
        if name.lower() in app.lower():
            return name
    return None


def post(server, body):
    req = urllib.request.Request(f"{server}/events", data=json.dumps(body).encode(),
                                 headers={"Content-Type": "application/json"})
    urllib.request.urlopen(req, timeout=3).read()


def session_state(server, session_id) -> str:
    with urllib.request.urlopen(f"{server}/sessions/{session_id}", timeout=3) as r:
        return json.load(r)["state"]


def main(argv=None):
    p = argparse.ArgumentParser(description="Tapat desktop watcher")
    p.add_argument("--email", required=True)
```

```
    p.add_argument("--session", required=True, help="session id from the exam page")
    p.add_argument("--token", required=True, help="session token from the exam pag
e")
    p.add_argument("--server", default="https://tapat-xrg5.onrender.com")
    p.add_argument("--apps", default=",".join(DEFAULT_APPS), help="comma-separated a
pp names to watch")
    a = p.parse_args(argv)
    watched = [x.strip() for x in a.apps.split(",") if x.strip()]
    last = None
    tick = 0
    print(f"Tapat watcher running for {a.email}. Watching: {', '.join(watched)}")
    while True:
        app = frontmost_app()
        hit = matches(app, watched) if app else None
        if hit and hit != last:
            try:
                post(a.server, {"session_id": a.session, "session_token": a.token,
"student": a.email, "type": "app_opened",
                                "detail": hit, "ts": time.time()})
            except OSError:
                pass
        last = hit
        tick += 1
        if tick % 5 == 0:
            try:
                if session_state(a.server, a.session) == "closed":
                    print("Session closed. Watcher exiting.")
                    return 0
            except OSError:
                pass
        time.sleep(1)


if __name__ == "__main__":
    sys.exit(main())
```

watcher/requirements.txt · Text · 2 lines

```
psutil; sys_platform == "win32"
pywin32; sys_platform == "win32"
```

watcher/README.md · Markdown · 13 lines

```
# Tapat watcher

Run during the quiz so Word, Notes, Preview and Acrobat opening gets logged.

```
python watcher.py --email you@school.edu --session SESSION_ID --token SESSION_TOKEN --server SERVER
```

- On the exam page, click **Copy watcher command** to get this line with your own values filled in.
- macOS uses `osascript`; Windows needs `pip install -r requirements.txt`.
- Logs app names only, never window titles.
- Exits by itself once the Form is submitted.
- Change the list with `--apps "Microsoft Word,Notes"`.
```

Appendix B4. Source code: Documentation build

1 files.

docs/build_pdf.py · Python · 182 lines

```python
"""Build docs/Tapat_INTE301_Project1_Lazaro.pdf from the README, screenshots and code.

Usage: python docs/build_pdf.py   (needs markdown, pygments, playwright, pypdf)
"""
import base64
import datetime
import html
import pathlib
import re

import markdown
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import TextLexer, get_lexer_for_filename
from pygments.util import ClassNotFound
from playwright.sync_api import sync_playwright
from pypdf import PdfReader, PdfWriter

ROOT = pathlib.Path(__file__).resolve().parent.parent
OUT = ROOT / "docs" / "Tapat_INTE301_Project1_Lazaro.pdf"
SHOTS = ROOT / "docs" / "screenshots"
ACCENT = "#1F3A8A"
FIGURES = [
    ("09_admin_quiz.png", "Admin quiz page on the live server: start link after Copy start link, a student with a blocked site and their timeline open, and a No session row after Import responses."),
    ("08_admin_login.png", "Professor log in. The password is set only in the server's environment."),
    ("10_admin_home.png", "Admin home: create a quiz from a Google Form link, and the list of quizzes."),
    ("02_popup.png", "Extension popup: the student pastes the start link and enters their school email."),
    ("02b_popup_waking.png", "Popup while the free server wakes up. It waits and opens the exam once the server answers."),
    ("03_consent.png", "Consent screen shown before the quiz starts."),
    ("04_exam.png", "Exam page with a sample quiz embedded in place of a real Google Form."),
    ("05_blocked.png", "Block page shown when chatgpt.com is opened during the quiz; the attempt is logged."),
    ("02c_popup_offline.png", "Popup while the server can't be reached: events are kept in the browser and sent in order later."),
    ("06_submitted.png", "Sample quiz submitted: Tapat detects the formResponse page and closes the session."),
]
PARTS = [
    ("Tapat server", ["server/app.py", "server/db.py", "server/auth.py", "server/seed.py", "server/run.sh",
                      "server/requirements.txt", "server/sample_responses.csv", "server/tests/test_app.py",
                      "render.yaml"]),
    ("Chrome extension", ["extension/manifest.json", "extension/config.js", "extension/background.js",
                          "extension/popup.html", "extension/popup.js", "extension/exam.html",
                          "extension/exam.js", "extension/blocked.html", "extension/blocked.js",
                          "extension/style.css", "extension/tests/blocklist.test.js"]),
    ("Desktop watcher", ["watcher/watcher.py", "watcher/requirements.txt", "watche
```

```
r/README.md"]),
 44      ("Documentation build", ["docs/build_pdf.py"]),
 45  ]
 46  LANG = {".py": "Python", ".js": "JavaScript", ".json": "JSON", ".html": "HTML", ".c
ss": "CSS",
 47          ".sh": "Bash", ".txt": "Text", ".csv": "CSV", ".md": "Markdown", ".yaml":
"YAML"}
 48
 49  CSS = f"""
 50  @page {{ size: A4; margin: 20mm 20mm 22mm 20mm; }}
 51  body {{ font-family: 'Inter','Helvetica Neue',Arial,sans-serif; font-size: 10.5pt;
line-height: 1.5; color: #111; }}
 52  h1, h2, h3 {{ color: {ACCENT}; break-after: avoid; }}
 53  a {{ color: {ACCENT}; }}
 54  code, pre {{ font-family: 'DejaVu Sans Mono',Consolas,monospace; font-size: 8.5pt;
}}
 55  pre {{ white-space: pre-wrap; word-break: break-all; background: #f5f7fb; padding:
8px; border-radius: 4px; }}
 56  table {{ border-collapse: collapse; width: 100%; margin: 8px 0; }}
 57  th, td {{ border: 1px solid #d5dbea; padding: 5px 7px; text-align: left; vertical-a
lign: top; }}
 58  th {{ background: #eef2fb; color: {ACCENT}; }}
 59  .page {{ break-before: page; }}
 60  .cover {{ text-align: center; padding-top: 18mm; }}
 61  .cover h1 {{ font-size: 44pt; margin: 0; }}
 62  .cover .one {{ font-size: 14pt; color: #333; margin: 6px 0 22px; }}
 63  .cover .meta {{ font-size: 11pt; line-height: 1.8; }}
 64  .cover img.logo {{ width: 95mm; border: 0; margin: 0 0 6px; }}
 65  .cover img:not(.logo) {{ max-width: 100%; max-height: 125mm; border: 1px solid #cc
d; margin-top: 18px; }}
 66  .toc ol {{ font-size: 12pt; line-height: 2; }}
 67  .readme {{ counter-reset: h2; }}
 68  .readme h1 {{ display: none; }}
 69  .readme h2.sec {{ break-before: page; }}
 70  .cover .kind {{ font-size: 13pt; letter-spacing: .2em; text-transform: uppercase; c
olor: {ACCENT}; margin: 4px 0 0; }}
 71  .cover .url {{ color: {ACCENT}; font-weight: 600; }}
 72  .toc .apx {{ font-size: 12pt; line-height: 2; margin: 6px 0 0 0; }}
 73  .toc ol.src {{ font-size: 11pt; line-height: 1.8; margin-left: 2em; list-style: non
e; }}
 74  .appendix h3 {{ color: {ACCENT}; }}
 75  .readme h2 {{ counter-increment: h2; border-bottom: 2px solid {ACCENT}; padding-bot
tom: 2px; }}
 76  .readme h2::before {{ content: counter(h2) ". "; }}
 77  figure {{ break-inside: avoid; margin: 0 0 14px; text-align: center; }}
 78  figure img {{ max-width: 100%; max-height: 105mm; border: 1px solid #ccd; }}
 79  figcaption {{ font-size: 9.5pt; color: #333; margin-top: 4px; }}
 80  .label {{ background: {ACCENT}; color: #fff; font-family: 'DejaVu Sans Mono',monosp
ace; font-size: 8.5pt; padding: 5px 8px; border-radius: 4px 4px 0 0; }}
 81  .listing table {{ border: 0; margin: 0; }}
 82  .listing td {{ border: 0; padding: 0; }}
 83  .listing .linenos {{ color: #999; padding-right: 8px; user-select: none; width: 1%;
}}
 84  .listing .linenos pre, .listing .code pre {{ margin: 0; border-radius: 0; padding:
6px; line-height: 1.35; }}
 85  .listing .linenos pre {{ background: #eef0f5; word-break: normal; white-space: pre;
}}
 86  .listing tr {{ break-inside: auto; }}
 87  .part h2 {{ font-size: 20pt; }}
 88  .listing .highlight pre {{ margin: 0; border-radius: 0 0 4px 4px; line-height: 1.3
5; }}
 89  .linenos {{ color: #999; background: #eef0f5; padding: 0 6px 0 2px; margin-right: 6
px; }}
```

```
 90  {HtmlFormatter(style="friendly").get_style_defs(".highlight")}
 91  """
 92
 93
 94  def img(name, folder=SHOTS):
 95      return "" + base64.b64encode((folder / name).read_bytes
()).decode()
 96
 97
 98  def manual_html():
 99      """docs/manual.md -> numbered sections, each starting on a new page, with numbe
red figures."""
100      md = (ROOT / "docs" / "manual.md").read_text()
101      body = markdown.markdown(md, extensions=["tables", "fenced_code"])
102      captions = dict(FIGURES)
103      counter = [0]
104
105      def figure(m):
106          name = m.group(1)
107          counter[0] += 1
108          cap = html.escape(captions[name])
109          return (f'<figure><img src="{img(name)}"><figcaption><b>Figure {counter
[0]}.</b> {cap}'
110                  f'</figcaption></figure>')
111
112      body = re.sub(r"<p>@@FIG:([\w.]+)@@</p>", figure, body)
113      body = re.sub(r"<h1>(.*?)</h1>", r'<h2 class="sec">\1</h2>', body)
114      sections = re.findall(r'<h2 class="sec">(.*?)</h2>', body)
115      deploy = (ROOT / "docs" / "deploy.md").read_text()
116      deploy = re.sub(r"^# .*\n", "", deploy).replace("\n## ", "\n### ")
117      appendix = ('<div class="page appendix"><h2>Appendix A. Deployment</h2>'
118                  + markdown.markdown(deploy, extensions=["tables", "fenced_code"]) +
"</div>")
119      return body, sections, appendix
120
121
122  def listing(path):
123      p = ROOT / path
124      src = p.read_text()
125      n = src.count("\n") + (0 if src.endswith("\n") else 1)
126      try:
127          lexer = get_lexer_for_filename(p.name, stripnl=False)
128      except ClassNotFound:
129          lexer = TextLexer(stripnl=False)
130      code = highlight(src, lexer, HtmlFormatter(linenos="inline", style="friendly"))
131      lang = LANG.get(p.suffix, "Text")
132      return f'<div class="page listing"><div class="label">{path} · {lang} · {n} lin
es</div>{code}</div>'
133
134
135  def build():
136      body, sections, appendix = manual_html()
137      today = datetime.date.today().strftime("%B %d, %Y").replace(" 0", " ")
138      toc = "".join(f"<li>{s}</li>" for s in sections)
139      toc += ('</ol><p class="apx">Appendix A. Deployment<br>Appendix B. Source code
</p><ol class=src>'
140              + "".join(f"<li>B{i + 1}. {t}</li>" for i, (t, _) in enumerate(PARTS)))
141      parts = "".join(
142          f'<div class="page part"><h2>Appendix B{i + 1}. Source code: {t}</h2><p>{le
n(files)} files.</p></div>'
143          + "".join(listing(f) for f in files)
144          for i, (t, files) in enumerate(PARTS)
145      )
```

```
    doc = f"""<!doctype html><html><head><meta charset="utf-8"><title>Tapat</title>
<meta name="author" content="Troy Lazaro"><style>{CSS}</style></head><body>
<section class="cover"><img class="logo" src="{img('logo.png', ROOT / 'logo')}" alt
="Tapat">
<p class="kind">User manual</p>
<p class="one">An exam monitor for Google Forms quizzes that blocks a listed set of
sites and logs when students leave the exam.</p>
<div class="meta"><b>INTE301 Systems Integration &amp; Architecture · Project 1</b>
<br>
Sir Rodel Villareaz<br>Troy Lazaro<br>{today}<br>
<span class="url">https://tapat-xrg5.onrender.com</span></div>
<img src="{img('09_admin_quiz.png')}"></section>
<section class="page toc"><h1>Contents</h1><ol>{toc}</ol></section>
<section class="readme">{body}</section>
{appendix}
{parts}
</body></html>"""
    (ROOT / "docs" / "build").mkdir(exist_ok=True)
    tmp = ROOT / "docs" / "build" / "tapat.html"
    tmp.write_text(doc)
    footer = ('<div style="width:100%;font-size:8pt;color:#555;padding:0 20mm;displ
ay:flex;'
              'justify-content:space-between;font-family:Arial"><span>Tapat · INTE3
01 Project 1</span>'
              '<span>Page <span class="pageNumber"></span> of <span class="totalPag
es"></span></span></div>')
    with sync_playwright() as pw:
        b = pw.chromium.launch()
        pg = b.new_page()
        pg.goto(tmp.as_uri())
        pg.pdf(path=str(OUT), format="A4", print_background=True, display_header_fo
oter=True,
               header_template="<div></div>", footer_template=footer,
               margin={"top": "20mm", "bottom": "22mm", "left": "20mm", "right": "2
0mm"})
        b.close()
    w = PdfWriter(clone_from=PdfReader(str(OUT)))
    w.add_metadata({"/Title": "Tapat", "/Author": "Troy Lazaro",
                    "/Subject": "INTE301 Systems Integration & Architecture · Proje
ct 1"})
    w.write(str(OUT))
    print(f"wrote {OUT}")


if __name__ == "__main__":
    build()
```